Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial 15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics

Master of Data Science

Graph Neural Networks for Homogeneous Graphs using Message Passing

---

THESIS to obtain the **DEGREE** of
MASTER OF DATA SCIENCE

A thesis presented by:
Rodrigo Emmanuel Macias Pantoja

Thesis Advisors:
Dr. Willie E. Coyote
Dr. Homer J. Simpson

Tlaquepaque, Jalisco, December, 2026

Instituto Tecnológico y de Estudios Superiores de Occidente

Reconocimiento de validez oficial de estudios de nivel superior según acuerdo secretarial 15018, publicado en el Diario Oficial de la Federación del 29 de noviembre de 1976.

Department of Mathematics and Physics
Master of Data Science Approval Form

Thesis Title: **Graph Neural Networks for Homogeneous Graphs using Message Passing**
Author: **Rodrigo Emmanuel Macias Pantoja**
Thesis Approved to complete all degree requirements for the Master of Science Degree in Data Science.

Thesis Advisor, **Dr. Willie E. Coyote**

Thesis Co-Advisor, **Dr. Homer J. Simpson**

Thesis Reader, **Dr. Frederick Flintstone**

Thesis Reader, **Dr. Farrokh Bulsara**

Academic Advisor, **Dr. Maryam Mirzajani**

Tlaquepaque, Jalisco, December, 2026

Graph Neural Networks for Homogeneous Graphs using Message Passing

Rodrigo Emmanuel Macias Pantoja

Abstract

Graph-structured data describes a domain as a set of entities together with the relations among them, a form that tabular, grid, and sequential representations do not capture: graphs vary in size, admit no canonical ordering of their elements, and connect each entity to a different number of others. A standard neural network, whose input dimension and coordinate ordering are fixed, cannot operate on such data without discarding the relational structure that carries much of the signal. Graph Neural Networks (GNNs) are neural network architectures designed to operate directly on graphs, respecting these properties instead of discarding them.

This work provides a self-contained reference on GNNs for homogeneous graphs under the message-passing framework. The foundations are established first: how graphs represent relational data, why the multilayer perceptron (MLP) fails on it, and the permutation invariance and equivariance that any layer defined on a graph must satisfy. The message-passing framework is then developed as the unifying formulation, decomposed into message, aggregation, and update functions, together with the receptive field that successive layers induce, the distinction between the transductive and inductive settings, and the ceiling on expressive power.

Three representative architectures are derived as instances of the framework, one for each standard level of graph learning: the Graph Convolutional Network (GCN) for node classification, the Graph Auto-Encoder (GAE) for link prediction, and the Graph Isomorphism Network (GIN) for graph classification. Each is implemented in PyTorch Geometric on a standard benchmark — Cora, Amazon Computers, and NCI1, respectively — and evaluated against an MLP baseline that predicts from node features alone; in every task the architecture that uses the graph improves on the baseline that does not. The limitations of the framework are examined last: oversmoothing, oversquashing, heterophily, and the bound on expressive power.

Graph Neural Networks for Homogeneous Graphs using Message Passing

Rodrigo Emmanuel Macias Pantoja

Resumen

Los datos estructurados como grafos describen un dominio como un conjunto de entidades junto con las relaciones entre ellas, una forma que las representaciones tabulares, en rejilla y secuenciales no capturan: los grafos varían en tamaño, no admiten un ordenamiento canónico de sus elementos y conectan cada entidad con un número distinto de otras. Una red neuronal estándar, cuya dimensión de entrada y ordenamiento de coordenadas son fijos, no puede operar sobre estos datos sin descartar la estructura relacional que porta buena parte de la señal. Las redes neuronales de grafos (GNN) son arquitecturas de redes neuronales diseñadas para operar directamente sobre grafos, respetando estas propiedades en lugar de descartarlas.

Este trabajo constituye una referencia autocontenida sobre las GNN para grafos homogéneos bajo el marco de paso de mensajes. Primero se establecen los fundamentos: cómo los grafos representan datos relacionales, por qué el perceptrón multicapa (MLP) fracasa ante ellos, y la invarianza y equivarianza a permutaciones que debe satisfacer toda capa definida sobre un grafo. A continuación, se desarrolla el marco de paso de mensajes como la formulación unificadora, descompuesto en las funciones de mensaje, agregación y actualización, junto con el campo receptivo que inducen capas sucesivas, la distinción entre los escenarios transductivo e inductivo, y el techo de la capacidad expresiva.

Se derivan tres arquitecturas representativas como instancias del marco, una por cada nivel estándar de aprendizaje sobre grafos: la Graph Convolutional Network (GCN) para clasificación de nodos, el Graph Auto-Encoder (GAE) para predicción de enlaces, y la Graph Isomorphism Network (GIN) para clasificación de grafos. Cada una se implementa en PyTorch Geometric sobre un conjunto de datos de referencia estándar — Cora, Amazon Computers y NCI1, respectivamente — y se evalúa frente a un modelo de referencia MLP que predice únicamente a partir de las características de los nodos; en todas las tareas la arquitectura que usa el grafo mejora a la que no lo usa. Por último, se examinan las limitaciones del marco: sobresuavizado (*oversmoothing*), sobrecompresión (*oversquashing*), heterofilia y la cota sobre la capacidad expresiva.

Contents

List of Figures

List of Tables

Dedicated to:

Dedicado a:

Escribir la dedicatoria pero en español.

1 Introduction

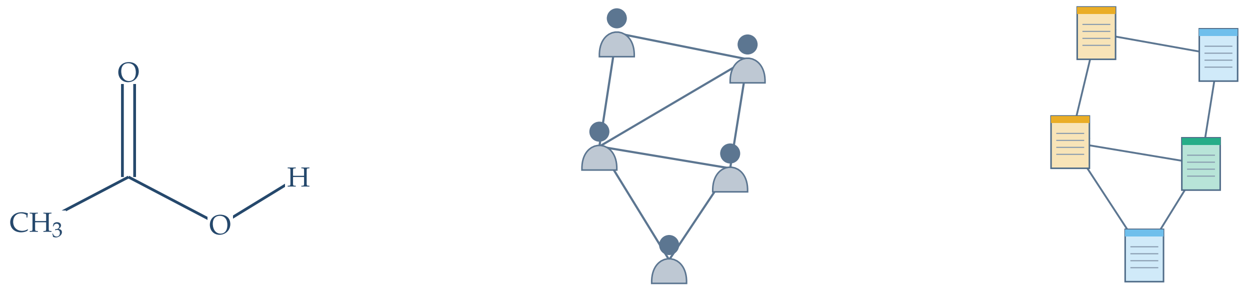
Contents

1.1 Learning from Graph-Structured Data

Data in machine learning often comes in a few familiar forms. Most datasets are tabular: a table of rows and columns in which each row is an independent example described by the same fixed set of features. Other data is organized as images, that is, grids of pixels, or as sequences such as text and audio, whose elements follow one another in a specific order. What these forms share is regularity: each has a fixed, well-defined structure that repeats across every example.

Many real-world systems do not fit any of these shapes. A molecule is a set of atoms connected by chemical bonds, and neither the number of atoms nor the way they are connected is the same from one molecule to the next. A social network is a set of people connected by relationships, where each person may be linked to a different number of others. A citation network is a set of papers that reference one another, each citing a different set of others. These are not tables, images, or sequences; they are networks of entities and the relations between them (Figure 1.1). Data of this kind is naturally described by a *graph*: a set of entities, called *nodes*, together with a set of connections between them, called *edges*. Both nodes and edges can carry their own data. For example, in a molecule, each node is an atom labeled with its element (such as carbon, hydrogen, or oxygen), and each edge is a bond of a particular type (such as single, double, or triple); in a social network, the nodes are people and the edges are the relationships among them.

Rather than treating a domain as a collection of isolated data points, it can be described in terms of entities and the relationships among them, which means there is an underlying graph whose structure ties



Molecule Social network Citation network

Figure 1.1: Three domains represented as graphs. A molecule, whose nodes are atoms labeled by their element and whose edges are bonds (single or double); a social network, whose nodes are people and whose edges are relationships; and a citation network, whose nodes are papers (colored by topic) and whose edges are citations between them. In each case the entities and their connections, rather than a table, grid, or sequence, define the structure of the data.

those entities together. Seen this way, the graph is not merely one possible encoding of a molecule or a social network but a faithful representation: it records exactly which entities exist and how they relate, without forcing the data into a shape it does not have. A wide range of domains exhibit this kind of relational structure, and modeling them explicitly as graphs, rather than ignoring the connections, can lead to more accurate predictions. Data organized in this way is called *graph-structured data*.

Most deep learning models are tailored to the regular structures described above. Graphs are harder to work with. They have no fixed size, since different graphs contain different numbers of nodes. They have no fixed layout: a pixel in a grid has a clear sense of place, with neighbors above, below, and to either side, whereas a node in a graph has no such spatial arrangement and may be connected to any number of others. And they have no natural ordering, with no first or last node and no canonical point from which to read the rest.

The goal, then, is a model that takes a graph as input and produces predictions about it, such as a label for each node, a connection between two nodes, or a single label for the graph as a whole. This raises the central question: how can a predictive model – concretely, a neural network – be designed to operate directly on a graph, respecting its variable size and its lack of order instead of discarding them? Neural networks built for this purpose are called *Graph Neural Networks*, hereafter abbreviated as GNNs.

1.2 *Objectives and Scope*

The general objective of this work is to provide a self-contained reference on GNNs for homogeneous graphs under the *message-passing* framework: a single document that develops the theory that motivates and defines these models, presents representative architectures, evaluates them through reproducible implementations, and serves as a foundational resource for studying the field.

Specifically, the work aims to:

- establish the foundations that motivate GNNs: how graphs represent relational data, why standard neural networks cannot learn from it directly, and the inductive biases a model on graphs must respect;
- develop the message-passing framework as the unifying formulation from which GNN architectures are derived;
- present representative architectures for the three standard learning tasks on graphs: the Graph Convolutional Network (GCN) for node classification, the Graph Auto-Encoder (GAE) for link prediction, and the Graph Isomorphism Network (GIN) for graph classification;
- implement the three architectures on standard benchmark datasets (Cora, Amazon Computers, and NCI1) using PyTorch Geometric, ensure reproducibility, evaluate each against a multilayer perceptron baseline, and examine the predictive benefit of explicitly modeling the relational structure of the data;
- examine the principal limitations of GNNs under this framework, including oversmoothing, oversquashing, heterophily, and the ceiling on their expressive power.

The graphs considered in this work are treated mainly as undirected, meaning their connections carry no direction, and simple, meaning that at most one connection joins any pair of nodes and that no node is connected to itself. This choice reflects simplicity rather than a limitation of the framework: message passing has enough design space to accommodate directed graphs, self-loops (a node connected to itself), and even multigraphs (more than one connection between the same pair of nodes).

The graphs considered in this work are also homogeneous: they contain a single kind of node and a single kind of connection. This restriction is deliberate. The homogeneous setting is where the message-passing framework is most directly understood, and it is the foundation on which more elaborate models are built, including heterogeneous graphs, which carry several kinds of nodes and connections and require machinery beyond the scope of this work.

The concepts introduced here are defined formally throughout the remainder of the document.

1.3 Structure of the Document

This introduction has set out the motivation, objectives, and scope of the work. The remainder of the document is organized as follows.

The first chapters establish the foundations. Chapter 2 surveys the tasks that graph learning addresses at the node, edge, and graph level, together with practical applications that motivate the rest of the work. Chapter 3 establishes the formal definitions and notation for graphs used throughout. Chapter 4 reviews the multilayer perceptron and shows why standard neural networks are ill-suited to relational data. Chapter 5 develops the notion of inductive bias and the permutation invariance and equivariance that any graph layer must satisfy.

Chapter 6 builds on these foundations to derive the message-passing framework: the message, aggregation, and update functions, the receptive field that successive layers induce, the expressive power of the resulting models, and the architectural design space it enables.

Chapter 7 presents three representative architectures within this framework: GCN for node classification, GAE for link prediction, and GIN for graph classification. Chapter 8 turns to practice, providing reproducible implementations of these architectures on the Cora, Amazon Computers, and NCI1 benchmarks in PyTorch Geometric, comparing each against a multilayer perceptron, with the code available in a companion repository.

Chapter 9 examines the limitations of GNNs under the message-passing framework, and Chapter 10 synthesizes the work and outlines directions for future research.

2 Learning Tasks on Graphs

Contents

Once a domain has been represented as graph-structured data, a natural question arises: what machine learning tasks can be defined on a graph? A graph can be viewed at three levels: its nodes, its edges, and the graph as a whole. Each gives rise to a family of prediction tasks: predicting a property of each node, predicting the existence or type of a connection between pairs of nodes, or predicting a property of the entire graph. These are known, respectively, as *node-level*, *edge-level*, and *graph-level tasks* (Figure 2.1), and they structure the architectures and implementations developed in this work.

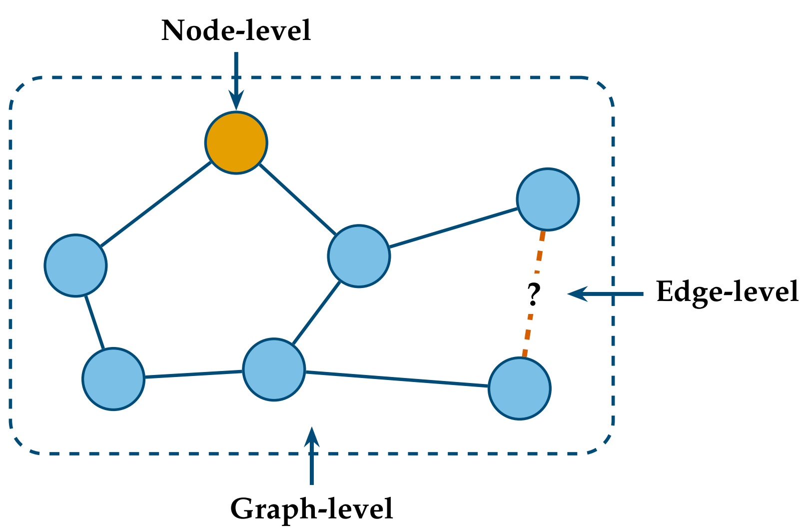


Figure 2.1: The three standard levels at which predictions are commonly made on a graph. A node-level task predicts a property of an individual node (highlighted); an edge-level task predicts a property of a connection between two nodes, such as whether it exists (dashed); and a graph-level task predicts a single property of the entire graph (framed).

These three levels correspond to the standard categories of graph learning tasks, but they do not exhaust the field. Other settings include

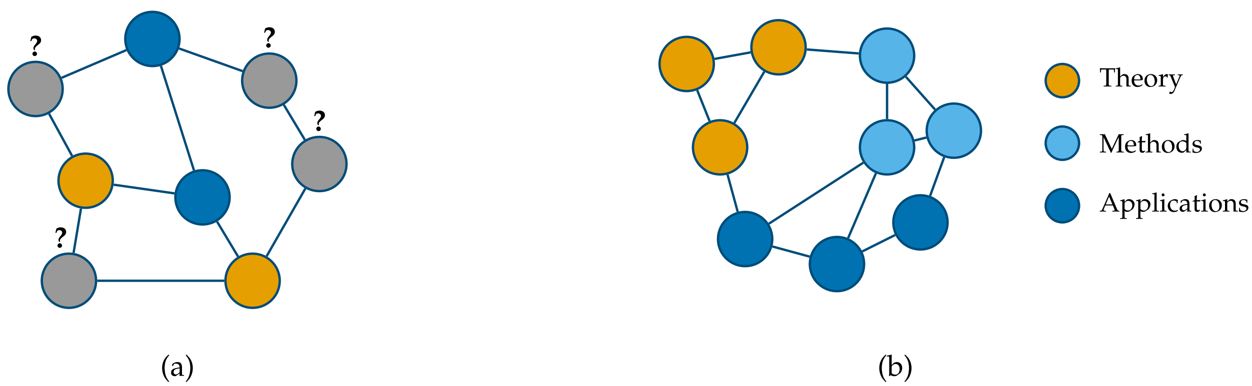


Figure 2.2: Left: the node classification task in the abstract. Some nodes carry a known label (shown by color) while others are unknown (marked with a question mark). Right: a concrete instance as a citation network, where each node is a scientific paper and the predicted label is its topic.

subgraph-level tasks, which focus on groups of related nodes, and *graph generation tasks*, which aim to generate new graphs, such as candidate molecular structures with desirable properties. Although these settings rely on the same representational principles introduced here, they lie outside the scope of this work, which focuses on the three standard task levels.

The following sections examine each of these levels in turn and present representative applications that motivate them.

2.1 Node-Level Tasks

A *node-level task* asks for a prediction about each individual node of a graph, drawing on both the node's own attributes and its position within the surrounding structure. The most common instance is *node classification*: assigning each node to one of a set of categories. A representative setting is a citation network, in which the nodes are scientific papers, the edges are citations between them, and the goal is to infer the topic of each paper (Figure 2.2). A paper's own text is informative, but so is its neighborhood: papers tend to cite others on related subjects, so the topics of a paper's neighbors are strong evidence for its own. Node classification exploits exactly this, combining what a node contains with what surrounds it.

The target need not be a category. A graph representation of a protein, for example, can be constructed where nodes represent amino acid residues and edges connect residues that are close in space. One may predict a discrete property of each residue, such as its functional role, or a continuous property, such as its position in three dimensions, a formulation that underlies recent advances in protein structure prediction [1].

[1] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873): 583–589, 2021. DOI: 10.1038/s41586-021-03819-2

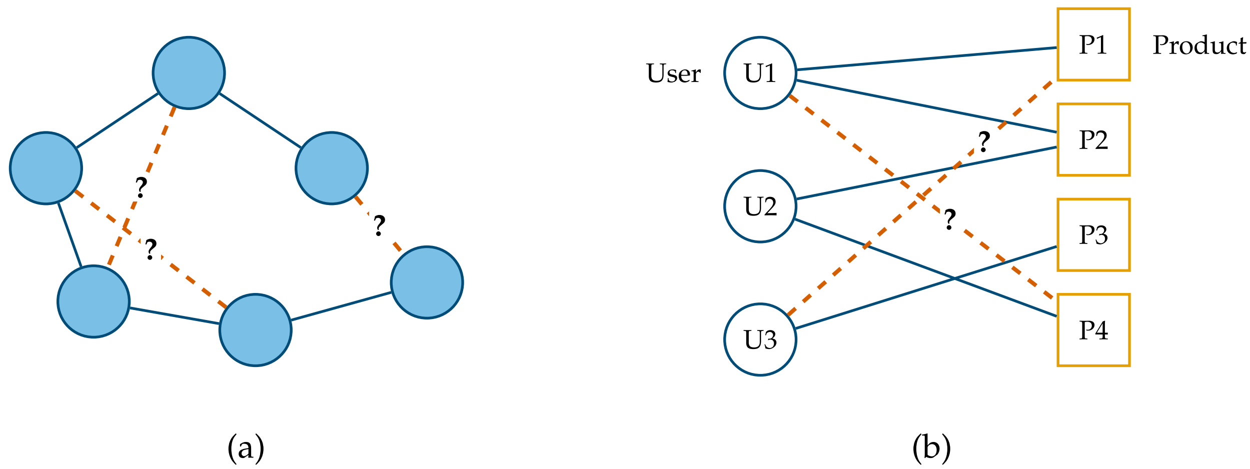


Figure 2.3: Left: the link prediction task in the abstract. Solid edges are observed connections; dashed edges marked with a question mark are candidate connections whose existence the model must predict. Right: a concrete instance as a recommendation setting, where the two kinds of entity (users, drawn as circles, and products, drawn as squares) are joined by observed interactions, and the task is to predict further user–product links.

The same framing extends to economic settings. In a financial network whose nodes are accounts and whose edges are transactions between them, fraud detection amounts to flagging each account as legitimate or fraudulent: a node classification problem in which fraudulent behavior is often more evident in the surrounding network of connections than in any single account considered in isolation.

2.2 *Edge-Level Tasks*

An *edge-level task* focuses on relationships between entities. The central example is *link prediction*: given the edges already present in a graph, predict which additional edges are likely to exist. Recommendation is one of its most familiar applications (Figure 2.3). In a product network, where nodes represent products and edges indicate co-purchases, predicting missing edges identifies items likely to be associated even if they have not yet been observed together. A richer formulation introduces users and products as distinct node types, with edges representing interactions such as purchases or ratings; link prediction then becomes the recommendation problem itself, anticipating which products a user is likely to prefer next.

What makes the graph formulation effective is that it learns from the relational structure rather than treating each entity in isolation. A recommender system that treats products only by their individual attributes, ignoring the network of interactions around them, discards precisely the signal that reveals which items behave similarly; methods that propagate information along the graph structure can outperform approaches that rely only on individual attributes, a principle exploited by graph-based recommender systems deployed at web scale [2].

[2] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890

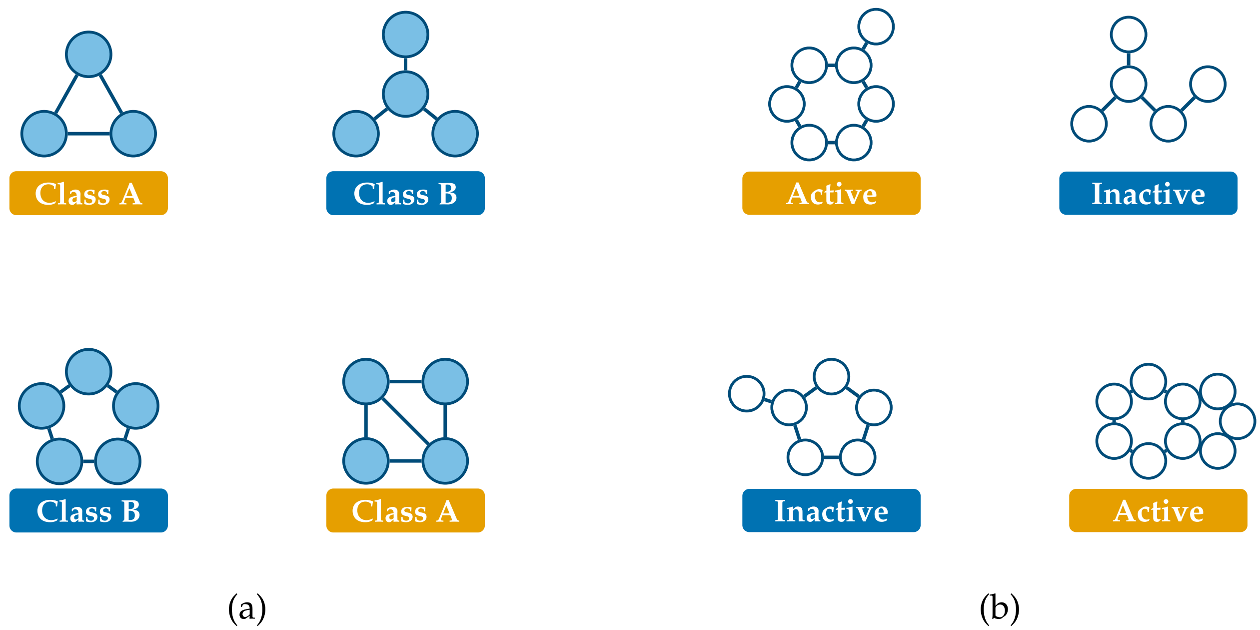


Figure 2.4: Left: the graph classification task in the abstract. The data is a collection of distinct graphs, differing in size and shape, each assigned a single label for the graph as a whole. Right: a concrete instance as molecular property prediction, where each graph is a molecule (atoms as nodes, bonds as edges) labeled by a property such as being active or inactive against a target.

Link prediction extends well beyond commerce. Given a pair of drugs, one may predict whether taking them together produces an adverse interaction, by placing both within a larger biological network and asking whether a particular kind of connection should join them [3]. The same formulation is also used to complete *knowledge graphs*, large networks of facts in which entities are connected by relations: many true relations are simply unobserved, and link prediction proposes the most plausible ones.

[3] Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294

2.3 *Graph-Level Tasks*

A *graph-level task* treats an entire graph as a single example and predicts a property of the whole. The representative task is *graph classification*: assigning a label to a complete graph (Figure 2.4). Here the data is not one large network but a collection of many separate graphs, each with its own label, and the model must summarize the structure and attributes of a graph into a single prediction. Molecular property prediction is the canonical example. A molecule is naturally a graph, with atoms as nodes and chemical bonds as edges, and prediction tasks about it are graph-level: whether a compound is toxic, or whether it is active against a given biological target. Because molecules vary in size and structure, this regime calls for a model able to handle graphs that vary in this way and return one prediction per graph.

This capability feeds directly into drug discovery, where large libraries of candidate compounds are screened to prioritize the few worth synthesizing and testing in the laboratory. A graph-level model trained to predict antibacterial activity, for instance, identified halicin, a compound structurally unlike conventional antibiotics and effective against resistant bacteria [4]. The same idea applies in materials science, where the atomic structure of a material is represented as a graph and a graph-level model predicts a physical property such as its stability or its mechanical strength.

Across all three levels, the graph is used in different ways, with the level of the task determining what is predicted and how a model must process the structure to predict it.

[4] Jonathan M. Stokes, Kevin Yang, Kyle Swanson, Wengong Jin, Andres Cubillos-Ruiz, Nina M. Donghia, Craig R. MacNair, Shawn French, Lindsey A. Carfrae, Zohar Bloom-Ackermann, Victoria M. Tran, Anush Chiappino-Pepe, Ahmed H. Badran, Ian W. Andrews, Emma J. Chory, George M. Church, Eric D. Brown, Tommi S. Jaakkola, Regina Barzilay, and James J. Collins. A deep learning approach to antibiotic discovery. *Cell*, 180(4):688–702, 2020. DOI: 10.1016/j.cell.2020.01.021

3 Fundamentals of Graph Theory

Contents

The applications surveyed in Chapter 2 share a single underlying object: the graph. To move from describing tasks to formulating them precisely, that object must itself be defined formally. This chapter introduces the formal language of graphs used throughout the rest of this work. It first defines graphs, neighborhoods, and node and edge features; it then encodes graph structure algebraically through the adjacency matrix and distinguishes the main types of graphs; it examines structural properties, from walks to isomorphism, on which later developments rely; it introduces the graph Laplacian and the spectral perspective it provides; and it closes with a summary of the notation fixed here.

3.1 *Formal Definitions*

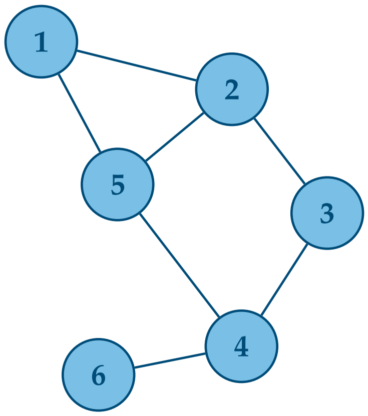
A *graph* is a pair

$$\mathcal{G} = (V, E), \tag{3.1}$$

where V is a finite set of *nodes*, also called vertices, and E is a set of *edges*, also called links, each one an unordered pair of nodes recording which two nodes are connected [1]. The number of nodes is denoted $n = |V|$ and the number of edges $m = |E|$. In classical graph-theoretic notation, generic nodes are denoted by $u, v \in V$, and an edge between them by the unordered pair $\{u, v\} \in E$; at this stage, the notation simply states that the two nodes are connected.

[1] Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5th edition, 2017. ISBN 978-3-662-53621-6

Because V and E are sets in the mathematical sense, neither carries an intrinsic order: a graph specifies which nodes are connected, and



(a)

	1	2	3	4	5	6
1	0	1	0	0	1	0
2	1	0	1	0	1	0
3	0	1	0	1	0	0
4	0	0	1	0	1	1
5	1	1	0	1	0	0
6	0	0	0	1	0	0

(b)

	1	2	3	4	5	6
1	0	1	1	0	0	1
2	1	0	0	1	1	0
3	1	0	0	0	0	0
4	0	1	0	0	1	0
5	0	1	0	1	0	1
6	1	0	0	0	1	0

(c)

Figure 3.1: A graph on six nodes (left) and its adjacency matrices under two different indexings of the nodes (center and right). The matrix on the right is obtained from the one in the center by the permutation $(1\,4)(2\,5)(3\,6)$, which exchanges nodes 1 and 4, 2 and 5, and 3 and 6. Both matrices describe the same graph: reordering the nodes permutes the rows and columns of $\mathbf{A}$.

nothing more. It is nevertheless convenient, particularly when graphs are represented by matrices, to fix an indexing of the nodes by integers. Throughout this work, nodes are therefore denoted $i, j \in \{1, \ldots, n\}$, with the convention that i refers to the node under consideration and j to a neighboring node whenever a neighborhood relation is involved. The choice of indexing is arbitrary and carries no information about the graph itself, a point this chapter returns to.

Two nodes are *adjacent*, or *neighbors*, when an edge connects them. The *neighborhood* of a node i is the set of its neighbors,

$$\mathcal{N}(i) = \{\, j \in V : \{i, j\} \in E \,\}, \tag{3.2}$$

and the *degree* of node i is the number of neighbors it has,

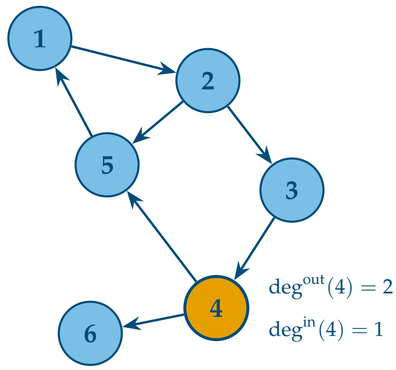
$$\deg(i) = |\mathcal{N}(i)|. \tag{3.3}$$

In a social network, for example, the neighborhood of a person is the set of people connected to them, and the degree counts them.

Beyond its structure, a graph typically carries data [2]. Each node i has an associated *node feature vector* $\mathbf{x}_i \in \mathbb{R}^d$, where d is the node feature dimension, describing the attributes of the entity the node represents, such as the textual content of a paper in a citation network, or the chemical element of an atom in a molecule. Stacking the n node feature vectors as rows produces the *node feature matrix*

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \tag{3.4}$$

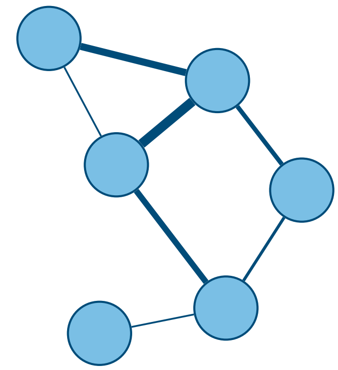
[2] William L. Hamilton. *Graph Representation Learning*, volume 46 of *Synthesis Lectures on Artificial Intelligence and Machine Learning*. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6



(a)

	1	**2**	**3**	**4**	**5**	**6**
1	0	1	0	0	0	0
2	0	0	1	0	1	0
3	0	0	0	1	0	0
4	0	0	0	0	1	1
5	1	0	0	0	0	0
6	0	0	0	0	0	0

(b)



(c)

Figure 3.2: Left: a directed version of the graph of Figure 3.1 together with its adjacency matrix (center). Edge orientation makes $\mathbf{A}$ asymmetric: row i lists the targets of the edges leaving node i, and the highlighted node is annotated with its in-degree and out-degree. Right: a weighted graph, where edge thickness is proportional to the weight w_{ij}.

whose i-th row is the transposed feature vector of node i. Writing the features as a matrix presupposes that the node indexing is fixed as above. Edges can carry attributes as well: the *edge feature vector* $\mathbf{e}_{i,j} \in \mathbb{R}^c$, where c is the edge feature dimension, describes the connection between nodes i and j, such as the type of a chemical bond in a molecule.

3.2 The Adjacency Matrix and Graph Types

With the nodes indexed from 1 to n, the structure of a graph can be encoded in a single matrix. The *adjacency matrix* of $\mathcal{G}$ is the matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ with entries

$$A_{ij} = \begin{cases} 1 & \text{if } \{i,j\} \in E, \\ 0 & \text{otherwise,} \end{cases} \tag{3.5}$$

so that each entry records the presence or absence of a single edge.

The adjacency matrix depends on the node indexing. Reordering the nodes permutes the rows and columns of $\mathbf{A}$, producing a different matrix that describes exactly the same graph. Figure 3.1 shows a graph together with its adjacency matrices under two different indexings: a graph has not one adjacency matrix but one per indexing, and no single one of them is privileged.

There are $n!$ possible indexings of n nodes, and each yields a valid adjacency matrix of the same graph.

The first distinction among graph types concerns the orientation of edges. In an *undirected* graph, the setting assumed so far, an edge $\{i,j\}$ has no orientation and can be traversed in either direction; its adjacency matrix is consequently symmetric, $\mathbf{A} = \mathbf{A}^\top$. In a *directed* graph, edges are instead ordered pairs: the pair $(i,j) \in E$ denotes an edge from a *source* node i to a *target* node j, and the presence of (i,j) no longer implies the presence of (j,i). The adjacency entry reads $A_{ij} = 1$ exactly

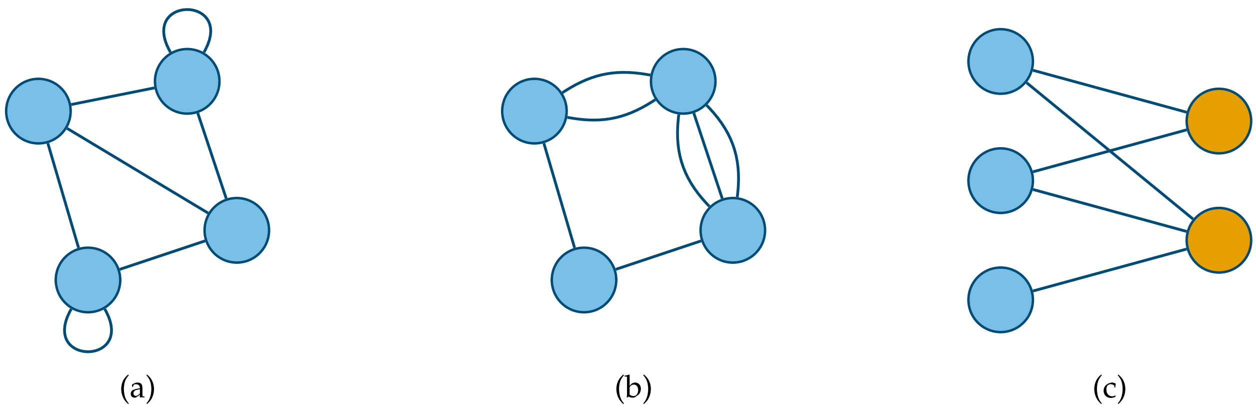


Figure 3.3: Left: a graph with self-loops. Center: a multigraph with parallel edges. Right: a bipartite graph, with its two subsets of nodes drawn in distinct colors.

when $(i, j) \in E$, so $\mathbf{A}$ is in general not symmetric and row i lists the targets of the edges leaving node i. The degree of a node accordingly splits in two: its *out-degree*, the number of edges leaving it, and its *in-degree*, the number of edges arriving at it.

A second distinction concerns whether edges carry a magnitude. In an *unweighted* graph, every edge counts the same and (3.5) applies as stated. In a *weighted* graph, each edge has an associated scalar weight $w_{ij} \in \mathbb{R}$, expressing, for instance, the strength of a relationship or a physical distance; the adjacency matrix then stores $A_{ij} = w_{ij}$ in place of 1 (Figure 3.2). A scalar weight can be viewed as the special case $c = 1$ of the edge feature vectors introduced in Section 3.1, that is, a single number attached to each edge.

A *self-loop* is an edge connecting a node to itself; in the adjacency matrix it appears as a nonzero diagonal entry, $A_{ii} \neq 0$. A *multigraph* allows several parallel edges between the same pair of nodes, a multiplicity that the binary entries of (3.5) cannot record. A graph with no self-loops and at most one edge between any pair of nodes is called *simple*. A *bipartite* graph, in contrast, is not a departure from simplicity but a structural subclass of it: its nodes partition into two disjoint subsets, and every edge connects a node of one subset with a node of the other, as in a network of users and products joined by purchases. A graph with this structure is always identified as bipartite, so that a graph described only as simple implies no such partition. Figure 3.3 illustrates the three cases.

The last distinction concerns the kinds of entities and relations a graph contains. A graph is *homogeneous* when all of its nodes represent one kind of entity and all of its edges represent one kind of relation: every node then shares the same feature space $\mathbb{R}^d$ and every edge the same feature space $\mathbb{R}^c$. A citation network is homogeneous, since every node is a paper and every edge a citation. A graph is *heterogeneous* when it contains several types of nodes or edges, each with its own semantics and, typically, its own feature space [3]. A citation network that includes both papers and their authors is heterogeneous: papers and authors are different kinds of entities, joined by a relation of authorship. Figure 3.4 contrasts the two settings.

[3] William L. Hamilton. *Graph Representation Learning*, volume 46 of *Synthesis Lectures on Artificial Intelligence and Machine Learning*. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

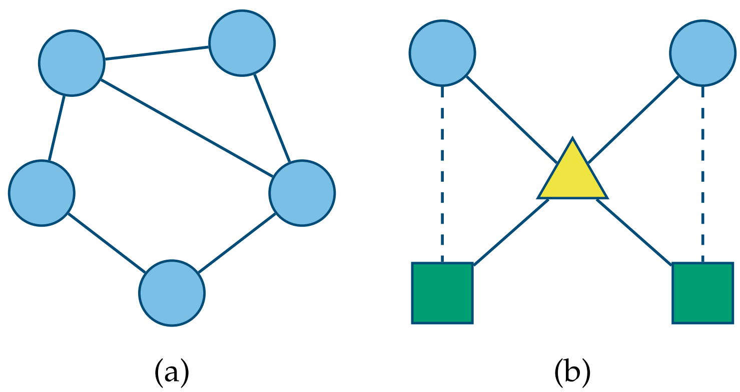


Figure 3.4: Left: a homogeneous graph, in which all nodes represent the same kind of entity and all edges the same relation. Right: a heterogeneous graph, in which node shapes denote different entity types and edge styles denote different relations.

As established in Chapter 1, the graphs treated in this work are undirected, simple, and homogeneous.

3.3 Walks, Connectivity, Homophily, and Isomorphism

The definitions so far describe a graph locally: nodes, their neighbors, and their attributes. Several properties that matter in the remainder of this work are instead structural properties that extend beyond an individual node, arising from how edges chain together across the graph. A *walk* is a sequence of nodes in which each consecutive pair is joined by an edge, and its *length* is the number of edges traversed; a walk may revisit nodes and edges. A *path* is a walk in which all nodes are distinct (Figure 3.5).

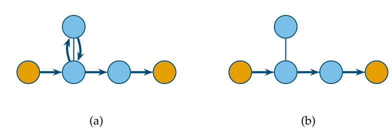


Figure 3.5: A walk that revisits a node (left) and a shortest path between the same endpoints (right), whose length is their geodesic distance.

The *geodesic distance* between two nodes is the length of a shortest path between them,

$$d(i, j) = \min\Big\{k \in \mathbb{N} : \text{there is a path of length } k \text{ between } i \text{ and } j\Big\} \tag{3.6}$$

with $d(i, i) = 0$. The distance is always written with its two arguments, $d(i, j)$, which distinguishes it from the node feature dimension d. When no path joins i and j, the distance is taken to be infinite, a situation characterized next.

A graph is *connected* when a path exists between every pair of its nodes, that is, when $d(i, j)$ is finite for all $i, j \in V$. When it is not, the graph decomposes into *connected components*: maximal groups of nodes that are mutually reachable, with no edges between groups (Figure 3.6). In directed graphs, connectivity refines into weak and strong variants according to whether edge orientations are respected; since this work treats undirected graphs, the refinement is not developed further.

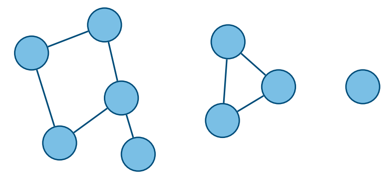


Figure 3.6: A disconnected graph with three connected components: one of five nodes, one of three nodes, and a single isolated node.

Structure and attributes are often correlated. A graph exhibits *homophily* when adjacent nodes tend to have similar attributes, and *heterophily* when they tend to differ. The notion originates in the analysis of social networks, where similar individuals form ties at higher rates than dissimilar ones [4], and it extends to many of the graphs of Chapter 2: in a citation network, a paper predominantly cites papers on related topics, so neighboring nodes tend to share a subject. Whether a graph is homophilous or heterophilous is an empirical property of the data rather than a structural constraint, and the degree to which it holds varies from one graph to another.

[4] Miller McPherson, Lynn Smith-Lovin, and James M. Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27(1):415–444, 2001. DOI: 10.1146/annurev.soc.27.1.415

The arbitrariness of node indexing observed in Section 3.2 raises a final question: when do two graphs that are presented differently have the same structure? Two graphs $\mathcal{G}_1 = (V_1, E_1)$ and $\mathcal{G}_2 = (V_2, E_2)$ are *isomorphic* when there exists a bijection $\pi : V_1 \to V_2$ that preserves adjacency,

$$\{i, j\} \in E_1 \iff \{\pi(i), \pi(j)\} \in E_2, \tag{3.7}$$

for all nodes $i, j \in V_1$. Isomorphic graphs are the same graph up to a relabeling of its nodes; the two adjacency matrices of Figure 3.1 are representations related in exactly this way. Two graphs drawn very differently may be isomorphic (Figure 3.7). Determining whether two graphs are isomorphic is a fundamental problem in graph theory. A classical heuristic, the Weisfeiler–Leman test, distinguishes many pairs of non-isomorphic graphs in practice [5], and it reappears later in this work in connection with the expressive power of GNNs.

[5] Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M. Kriege, Martin Grohe, Matthias Fey, and Karsten Borgwardt. Weisfeiler and Leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023. https://jmlr.org/papers/v24/22-0240.html

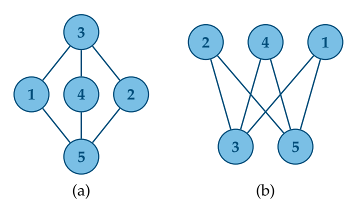


Figure 3.7: The same graph drawn in two different ways (left and right). A bijection between their node sets preserves all adjacencies.

3.4 The Graph Laplacian and the Spectral Perspective

The structural properties of Section 3.3 and the algebraic encoding of Section 3.2 meet in a single observation: powers of the adjacency matrix count walks. For any positive integer k, the entry in row i and column j of the k-th matrix power gives the number of walks of length k between the two nodes,

$$(\mathbf{A}^k)_{ij} = \big|\{\text{ walks of length } k \text{ between } i \text{ and } j\,\}\big|, \tag{3.8}$$

a count that is symmetric in i and j for the undirected graphs treated here. In particular, the entry is nonzero exactly when the two nodes are joined by some walk of that length, which ties the reachability notions of Section 3.3 directly to matrix algebra.

Another matrix derived from the adjacency matrix is the *degree matrix* $\mathbf{D} \in \mathbb{R}^{n \times n}$, the diagonal matrix that holds the node degrees,

$$\mathbf{D} = \operatorname{diag}\big(\deg(1), \ldots, \deg(n)\big), \tag{3.9}$$

with every off-diagonal entry equal to zero. Combining it with the adjacency matrix yields the central operator of spectral graph theory [6], the *graph Laplacian*

$$\mathbf{L} = \mathbf{D} - \mathbf{A}. \tag{3.10}$$

[6] Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8

Each diagonal entry of $\mathbf{L}$ is the degree of the corresponding node, and each off-diagonal entry is -1 where an edge is present and 0 otherwise. For the six-node graph of Figure 3.1, under the node indexing of its center panel and with $\mathbf{D} = \operatorname{diag}(2, 3, 2, 3, 3, 1)$, the Laplacian is

$$\mathbf{L} = \begin{bmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}.$$

The Laplacian admits a direct interpretation in terms of how values spread over a graph. Assigning a real number to each node defines a vector $\mathbf{f} \in \mathbb{R}^n$ called a *graph signal*, with component f_i the value at node i. The Laplacian measures how much such an assignment varies across edges through the quadratic form

$$\mathbf{f}^\top \mathbf{L} \mathbf{f} = \sum_{\{i,j\} \in E} (f_i - f_j)^2, \tag{3.11}$$

which sums the squared difference between the values at the two endpoints of every edge. The form vanishes when $\mathbf{f}$ is constant on each connected component and grows as the values assigned to neighboring nodes diverge. Because it is a sum of squares, the quadratic form is nonnegative for every $\mathbf{f}$, so the Laplacian is *positive semidefinite*.

Being real and symmetric, the Laplacian admits an eigendecomposition

$$\mathbf{L} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top, \tag{3.12}$$

where the columns of $\mathbf{U} \in \mathbb{R}^{n \times n}$ form an orthonormal basis of eigenvectors and $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ collects the corresponding eigenvalues. Symmetry guarantees that these eigenvalues are real, while positive semidefiniteness guarantees that they are nonnegative; ordered, they satisfy $0 = \lambda_1 \leq \cdots \leq \lambda_n$. The smallest eigenvalue is always zero, with the constant vector $\mathbf{1}_n$ as an eigenvector, since the rows of $\mathbf{L}$ sum to zero. More can be said: the multiplicity of the eigenvalue 0 equals the number of connected components of the graph, linking the spectral description back to the connectivity of Section 3.3.

The *symmetric normalized Laplacian* is obtained by rescaling the Laplacian with the node degrees,

$$\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2}\mathbf{L}\mathbf{D}^{-1/2} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}, \tag{3.13}$$

where $\mathbf{I}$ is the identity matrix and $\mathbf{D}^{-1/2}$ is the diagonal matrix of inverse square-root degrees. It is again symmetric and positive semidefinite, so it shares the spectral structure just described, while its eigenvalues are confined to the interval $[0, 2]$ [7]. Together, these properties make the Laplacian a natural bridge between the structure of a graph and the linear algebra used to operate on it.

For a node of degree zero the corresponding entry of $\mathbf{D}^{-1/2}$ is taken to be zero.

[7] Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8

3.5 Notation Summary

The notation introduced in this chapter is collected in Table 3.1 for reference. It is used consistently throughout the remainder of this work.

Symbol	Meaning	Reference
$\mathcal{G} = (V, E)$	Graph with node set V and edge set E	(3.1)
$n,\ m$	Number of nodes and edges	§3.1
$i,\ j$	Nodes; i central, j a neighbor	§3.1
$\mathcal{N}(i)$	Neighborhood of node i	(3.2)
$\deg(i)$	Degree of node i	(3.3)
$d(i, j)$	Geodesic distance between i and j	(3.6)
π	Isomorphism bijection between node sets	(3.7)
$\mathbf{x}_i,\ d$	Node feature vector and its dimension	(3.4)
$\mathbf{X}$	Node feature matrix	(3.4)
$\mathbf{e}_{i,j},\ c$	Edge feature vector and its dimension	§3.1
w_{ij}	Scalar edge weight	§3.2
$\mathbf{A}$	Adjacency matrix	(3.5)
$(\mathbf{A}^k)_{ij}$	Walks of length k between i and j	(3.8)
$\mathbf{D}$	Degree matrix	(3.9)
$\mathbf{L}$	Graph Laplacian	(3.10)
$\mathbf{L}_{\text{sym}}$	Symmetric normalized Laplacian	(3.13)
$\mathbf{f}$	Graph signal, one real value per node	(3.11)
$\mathbf{U},\ \boldsymbol{\Lambda}$	Eigenvectors and eigenvalues of $\mathbf{L}$	(3.12)
$\lambda_1, \ldots, \lambda_n$	Laplacian eigenvalues, in increasing order	(3.12)

Table 3.1: Notation introduced in this chapter, with the equation or section in which each symbol is defined.

4 Neural Networks and Their Limitations on Relational Data

Contents

Before constructing a model that operates on graphs, it is necessary to examine the foundational model of deep learning: the multilayer perceptron. This chapter first assembles the multilayer perceptron and the procedure by which it is trained, so that it stands as a complete and well-defined object. It then attempts to apply this model to a graph and shows that the attempt fails. The reasons for that failure are not incidental: they identify precisely the properties a model must possess to operate on graphs.

4.1 The Multilayer Perceptron

The *multilayer perceptron* (MLP) is the elementary neural network. It maps a fixed-size input vector to an output vector through a sequence of layers, each applying a linear transformation followed by a nonlinear function [1].

A single layer takes a vector and produces another vector. Writing $\mathbf{h}^{(l-1)}$ for the input to layer l and $\mathbf{h}^{(l)}$ for its output, the layer computes

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \tag{4.1}$$

where $\mathbf{W}^{(l)}$ is a learnable weight matrix, $\mathbf{b}^{(l)}$ a learnable bias vector, and σ the *activation function*, a fixed nonlinear function applied elementwise. The index $l = 1, \ldots, L$ runs over the L layers of the network. The weight matrix and bias vector of each layer are the *parameters* of the model, collectively denoted by θ, and are the quantities adjusted during training.

[1] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

The activation function σ is what gives the network its expressive power. Without it, the composition of linear layers would itself be a single linear transformation, no matter how many layers were stacked. A nonlinear σ — for instance the rectified linear unit (ReLU), $\sigma(x) = \max(0, x)$ — allows the network to represent nonlinear relationships between input and output [2].

[2] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

A multilayer perceptron is the composition of several such layers. Taking the input vector $\mathbf{x}$ as the layer-zero representation, $\mathbf{h}^{(0)} = \mathbf{x}$, the network applies the layers in sequence,

$$f(\mathbf{x}) = f^{(L)} \circ \cdots \circ f^{(1)}(\mathbf{x}), \tag{4.2}$$

where each $f^{(l)}$ denotes the transformation of layer l defined in (4.1), and the output of the final layer is the prediction of the network. The layers between the input and the output are the *hidden layers*, and the number of layers L is the *depth* of the network. The map f takes a fixed-size vector $\mathbf{x} \in \mathbb{R}^d$ and returns an output of fixed size, determined by the dimensions of the weight matrices (Figure 4.1).

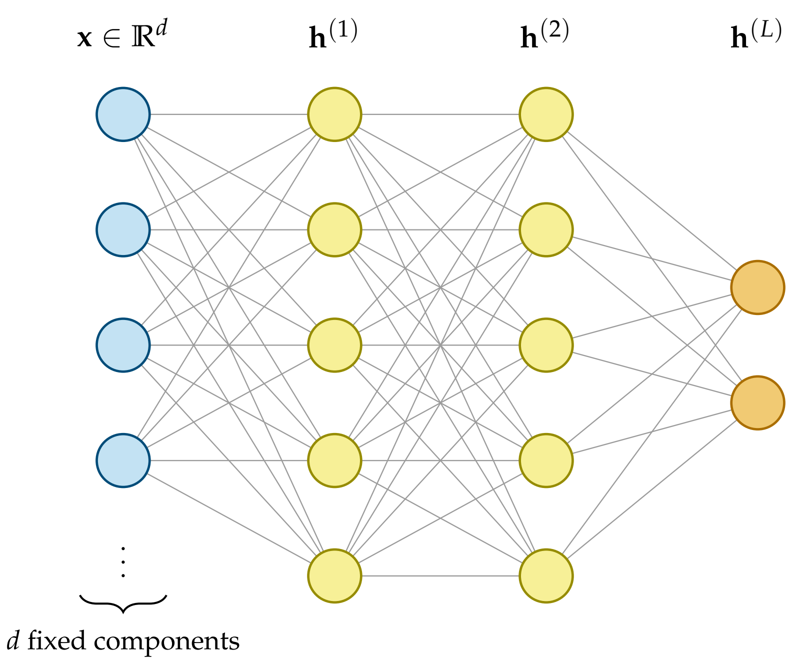


Figure 4.1: A multilayer perceptron mapping a fixed-size input vector $\mathbf{x} \in \mathbb{R}^d$ through two hidden layers to an output. Each layer applies a learnable linear transformation followed by an elementwise nonlinearity. The dimension of the input is fixed in advance by the architecture: every input the network processes must be a vector of exactly d components.

This last property is essential to what follows. The architecture fixes the size of the input in advance: the product $\mathbf{W}^{(1)}\mathbf{x}$ is defined only if the input vector $\mathbf{x}$ has the required dimension, and the network assumes that its components are arranged in a definite order. A multilayer perceptron is a function on $\mathbb{R}^d$ for a fixed d, and on nothing else.

4.2 Training by Gradient Descent

The parameters θ are not set by hand but learned from data, by adjusting them so that the network's predictions match ground-truth targets. This requires a measure of how wrong the predictions are and a procedure for reducing it.

The measure is a *loss function*. Given a dataset of n examples with inputs $\mathbf{x}_i$ and targets y_i, the loss aggregates the discrepancy between each prediction $f(\mathbf{x}_i)$ and its target into a single scalar,

$$\mathcal{L}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(\mathbf{x}_i), y_i)\,, \tag{4.3}$$

where ℓ is a per-example loss, such as the squared error for regression or the cross-entropy for classification. The loss is a function of the parameters θ, since the prediction $f(\mathbf{x}_i)$ depends on them; training is the search for the θ that minimizes $\mathcal{L}(\theta)$.

That search is carried out by *gradient descent*. The gradient $\nabla_\theta \mathcal{L}$ points in the direction of steepest increase of the loss; moving the parameters in the opposite direction decreases it. Each step updates the parameters by

$$\theta \leftarrow \theta - \eta\, \nabla_\theta \mathcal{L}(\theta), \tag{4.4}$$

where the scalar $\eta > 0$ is the *learning rate*, controlling the size of each step. The gradient itself is computed by *backpropagation*, the application of the chain rule across the layers of the network [3]; it yields the derivative of the loss with respect to every parameter, and is the algorithm that makes training deep networks practical.

[3] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

Computing the gradient in (4.4) over the full dataset is costly when n is large, since it requires a pass over every example for a single update. *Stochastic gradient descent* replaces the full-dataset gradient with an estimate computed on a small random subset of the data, a *minibatch* $\mathcal{B}$. The method was originally defined for a single example per step — the source of the name, as each step follows a noisy, *stochastic* estimate of the gradient rather than the exact one — but is generally applied over a minibatch, which averages the per-example gradients,

$$\nabla_\theta \mathcal{L}(\theta) \approx \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} \nabla_\theta\, \ell(f(\mathbf{x}_i), y_i)\,. \tag{4.5}$$

Under uniform random sampling of examples, the minibatch gradient is an unbiased estimate of the full gradient [4], and each update is far cheaper, so many more updates are performed for the same computational cost.

[4] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

The optimizer used throughout the implementations of this work is *Adam*, a method in the stochastic gradient descent family that adapts the effective step size for each parameter individually, using running estimates of the gradient. Its internal mechanics are not required for the developments that follow and are documented in the original work [5]. What matters is that the multilayer perceptron, with its parameters, loss, and training procedure, now stands as a complete and trainable object. The remainder of the chapter asks whether that object can be applied to a graph.

[5] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, 2015. https://arxiv.org/abs/1412.6980

4.3 *Why the Multilayer Perceptron Fails on Graphs*

The multilayer perceptron assembled above is a function on a fixed-size vector. A graph is not a vector. To apply the perceptron to a graph, the graph must first be converted into a vector of fixed size, and the difficulty is that no such conversion preserves what the graph encodes. This section attempts the conversion directly and examines what goes wrong; the failures that surface are the substance of the chapter.

Consider a node-level task: predicting a property of each node of a graph, as in the node classification setting of Chapter 2. There are two natural ways to present such a problem to a multilayer perceptron, and it is instructive that both fail, for different reasons.

The first approach attempts to encode the whole graph as a single input vector and feed it to the network. The structure of a graph on n nodes is carried by its adjacency matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, and its node attributes by the feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$. To form a single input vector, these matrices are flattened — their entries read out in order and laid end to end — and concatenated. This at once runs into two obstructions.

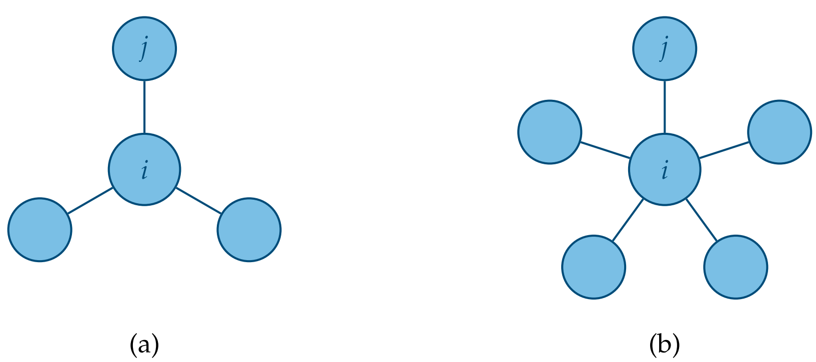


Figure 4.2: Two nodes with neighborhoods of different sizes: the node on the left has three neighbors, the node on the right has five. A model that processes a node together with its neighbors must accept inputs of varying size, which a multilayer perceptron cannot.

The first obstruction concerns size. The flattened adjacency matrix has n^2 entries and the flattened feature matrix has $n \times d$; the length of the resulting input vector therefore grows with the number of nodes and differs from one graph to another. But a multilayer perceptron, by (4.1), admits inputs of one fixed dimension only. A network built

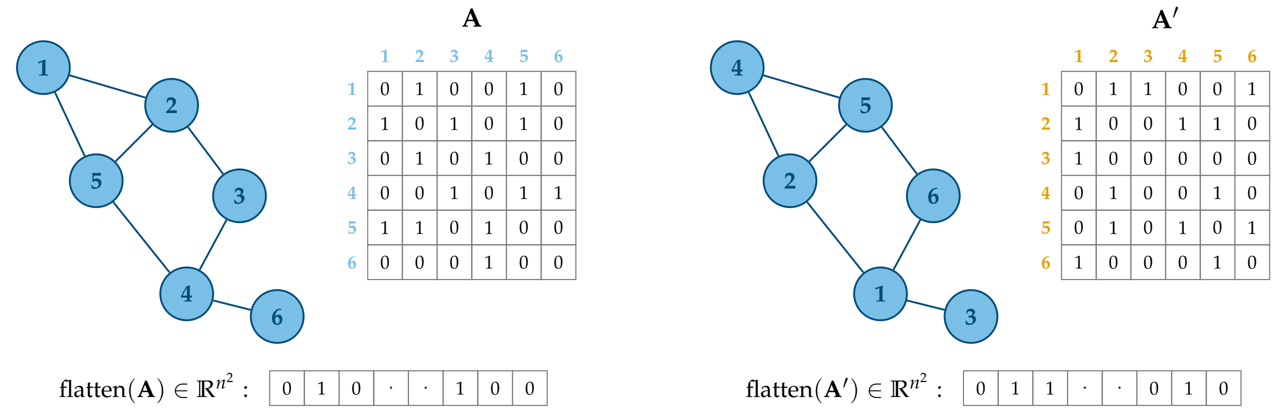


Figure 4.3: A single graph drawn twice with its nodes indexed in two different orders. Each ordering produces a different adjacency matrix, and flattening each matrix produces a different input vector. A multilayer perceptron receives the two vectors as unrelated inputs, although they encode the same graph.

for a graph of one size cannot accept a graph of another. The same obstruction arises even if one does not flatten the whole graph at once, but instead processes each node together with its neighbors: nodes differ in how many neighbors they have, so a neighborhood-based input has no fixed size either (Figure 4.2). This is the problem of *variable size*: neither across graphs nor across the nodes of a single graph is the number of entities fixed, whereas the multilayer perceptron demands a fixed-dimensional input.

The second obstruction concerns order. As established in Chapter 3, the nodes of a graph carry no intrinsic ordering; an indexing must be chosen to write down the adjacency matrix, and any of the $n!$ possible indexings describes the same graph. The two adjacency matrices of a graph under two different node orderings are different matrices, and flattening them yields two different vectors. A multilayer perceptron would receive these as two distinct inputs and could process them differently, even though they represent the same graph (Figure 4.3). The model would thus depend on an arbitrary choice that carries no information — the labeling of the nodes — and would have no means of recognizing that the choice is arbitrary. This is the problem of *node ordering*: the flattened representation imposes an order that the graph does not have.

Faced with these obstructions, one might abandon the adjacency matrix altogether and take the second approach: present each node to the network on its own, using only its feature vector $\mathbf{x}_i$, and predict the node's property from that. This input has a fixed size d, and the ordering of the nodes no longer enters, since each node is treated separately. Both size and ordering cease to be obstructions — but at a cost that defeats the purpose. With the adjacency matrix discarded, the structure of the graph never enters the computation at all. The network processes each node as an isolated example. Yet the tasks of Chapter 2 are relational by nature: in a citation network, a paper's topic is informed by the topics of the papers it cites, and a model that sees a node in isolation discards exactly this evidence. This is the problem of *missing relational structure*: the only input that fits a multilayer perceptron is one from which the relations have been removed.

The two most natural encodings thus fail in opposite ways. Including the adjacency matrix preserves the structure but breaks on variable size and node ordering; excluding it restores a fixed, order-free input but discards the structure entirely.

These failures reveal that the multilayer perceptron carries the wrong assumptions for relational data. Its *inductive bias* is that examples are independent and of fixed size, an assumption appropriate to vectors but false for graphs, whose entities are connected and whose representation admits no canonical order. What the analysis exposes is the assumptions a graph model must instead possess: it must accept inputs of varying size, it must be *invariant* to the ordering of the nodes, so that relabeling leaves its predictions unchanged, and it must let the structure of the graph participate in the computation. These three requirements, arrived at here by watching the multilayer perceptron fail, are the foundation on which the following chapters build a neural network suited to graphs.

5 *Inductive Bias, Invariance, and Equivariance*

Contents

The preceding analysis in Section 4.3 introduced two concepts in passing — the *inductive bias* of a model and the requirement that a graph model be *invariant* to node ordering — without developing either. This chapter develops both. It first makes precise what an inductive bias is and why successful generalization requires one, reading the inductive biases of successful architectures as evidence that each encodes the structure of its own domain. It then identifies the inductive bias appropriate to graphs and gives the two symmetry conditions that express it formally.

5.1 *Inductive Bias in Neural Architectures*

A model trained on a finite set of examples must, to be useful, make predictions on examples it has never seen. The training data alone cannot determine those predictions: infinitely many functions are consistent with any finite sample. A model generalizes only because it brings assumptions of its own about which functions are plausible before any data is observed. The set of such assumptions is the *inductive bias* of the model [1]. Generalization is therefore not possible without an inductive bias, and succeeds when that bias matches the structure of the problem.

[1] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3

Deep learning architectures differ primarily in the inductive biases their structure encodes. Three of them recur throughout this work: the multilayer perceptron, the convolutional neural network (CNN), and the recurrent neural network (RNN). From this point onward, these architectures are referred to exclusively by their acronyms, both in this chapter and throughout the remainder of the document.

The MLP, whose layers connect every input to every output, assumes that any input coordinate may interact with any other, that no particular structure among the coordinates is presumed, and that examples are treated independently. This is the appropriate bias when the input is an unstructured vector — such as tabular data — but the wrong bias for data with an underlying structure.

The CNN and the RNN illustrate how a well-matched bias is built into structure. The CNN, designed for data arranged on a regular grid such as the pixels of an image, applies the same local filter at every position. Two assumptions are encoded in this design. The first is *locality*: a pixel is informative about its immediate neighbors, so the filter reads only a small spatial window rather than the whole image. The second is that a pattern is meaningful regardless of where in the image it appears, so the same filter is reused at every position; a feature shifted across the grid produces the same feature shifted in the output. These assumptions make the CNN well suited to natural images. The RNN, designed for sequences, encodes a different bias: it processes elements in order, applying the same transition at every step. This assumes that the order of the elements is meaningful and that the same transformation can be reused across all positions in the sequence. These assumptions are appropriate for text and time series.

The same principle appears in all three. Each architecture succeeds when its inductive bias matches the structure of the data, and the match is built into how the architecture is wired rather than learned from data. The question this chapter must answer follows immediately: what is the structure of a graph, and what inductive bias matches that structure.

5.2 The Relational Inductive Bias

The defining structure of a graph is relational. An entity is not described in isolation but through its connections to other entities: in a citation network the subject of a paper is reflected in the papers it cites, and in a molecule the role of an atom is determined by the atoms bonded to it. The inductive bias appropriate to graphs is therefore the assumption that a node is characterized by its neighborhood — that the information relevant to a node resides in the node itself, in the nodes adjacent to it, and in the structure connecting them. This assumption is the *relational inductive bias* [2].

The relational inductive bias is the graph counterpart of the locality that the CNN encodes, and the comparison is exact in one respect and instructive in its difference. Both assume that an entity is informed by

[2] Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv preprint, 2018. URL https://arxiv.org/abs/1806.01261

what lies near it rather than by everything at once, and both read from a local region rather than the whole input. The difference lies in what counts as local. On a grid, locality is geometric: a pixel's neighbors are the pixels at fixed offsets around it, the same offsets everywhere, so a local window is defined once and applied uniformly. On a graph there are no fixed offsets and no uniform window. A node's neighborhood is whatever the edges make it, varying in size from one node to the next. The relational inductive bias retains locality while discarding the regular geometry that made it simple to express on a grid; locality on a graph is defined by adjacency, not by position.

Adjacency, however, fixes only which nodes are neighbors, not the order in which they are listed. A neighborhood is a set, and a set has no first element. The relational inductive bias says where a node's information comes from; it does not, and must not, depend on the order in which that information is enumerated, because the graph supplies no such order. Making this requirement precise calls for the concepts of the next two sections.

5.3 *Invariance and Equivariance*

The requirement that a model not depend on an arbitrary choice in its input is made precise by the notions of *invariance* and *equivariance*. Both describe how a function responds to transformations of its input, and they are defined for any function and any transformation before being specialized to graphs.

Let f be a function and let g denote a transformation applied to its input. The function f is *invariant* to g when transforming the input leaves the output unchanged:

$$f(g \cdot \mathbf{x}) = f(\mathbf{x}). \tag{5.1}$$

The transformation is applied to the input, and the output does not move. The function f is *equivariant* to g when transforming the input transforms the output in the same way:

$$f(g \cdot \mathbf{x}) = g \cdot f(\mathbf{x}). \tag{5.2}$$

The transformation is applied to the input, and the output moves in step with the input.

The distinction is between discarding a transformation and preserving it. Invariance identifies transformed and untransformed inputs by mapping them to the same output, so the transformation is discarded by f. Equivariance preserves the effect of the transformation: it survives the application of f and acts on the result, so applying f and then transforming gives the same answer as transforming and then applying f. Which of the two is required is not a matter of preference but of the task: when the quantity to be predicted is itself unaffected by the transformation, f must be invariant; when the quantity to be predicted is attached to parts of the input that the transformation moves, f must be equivariant, so that the prediction moves with the part it describes.

5.4 Permutation Invariance and Equivariance on Graphs

The transformation that matters for graphs is the reordering of nodes. As established in Chapter 3, the nodes of a graph carry no intrinsic order; an indexing must be chosen to write the adjacency matrix $\mathbf{A}$ and the feature matrix $\mathbf{X}$, and any of the $n!$ possible indexings describes the same graph. A reordering of the nodes is represented by a *permutation matrix* $\mathbf{P} \in \{0,1\}^{n\times n}$, a square matrix with exactly one entry equal to 1 in each row and each column and zeros elsewhere. An entry $P_{ij} = 1$ indicates that the node originally at index j is mapped to the new index i. Multiplying by $\mathbf{P}$ rearranges the rows of a matrix according to this relabeling. A permutation matrix is orthogonal,

$$\mathbf{P}\mathbf{P}^\top = \mathbf{I}, \tag{5.3}$$

which states that the reordering is invertible and that its inverse is its transpose.

Applying the reordering $\mathbf{P}$ to a graph acts on both of its matrices. The feature matrix becomes $\mathbf{PX}$, which moves each node's feature vector to the row assigned by the reordering. The adjacency matrix becomes $\mathbf{PAP}^\top$, with $\mathbf{P}$ reordering its rows and $\mathbf{P}^\top$ its columns, so that the entry recording an edge between two nodes follows them to their new positions. The pair $(\mathbf{PAP}^\top, \mathbf{PX})$ is the same graph as $(\mathbf{A}, \mathbf{X})$ under a different indexing; no edge has been added or removed, and no feature has changed.

A function defined on graphs must respect this fact, and which condition it must satisfy depends on the level of the task. For a task whose output describes the graph as a whole — a single prediction for the entire graph, as in graph classification — the output must not depend on the indexing at all. Such a function f must be *permutation invariant*:

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = f(\mathbf{A}, \mathbf{X}). \tag{5.4}$$

Reordering the nodes leaves the graph-level prediction unchanged. For a task whose output is attached to the individual nodes — one prediction per node, as in node classification — the output must follow the nodes when they are reordered. In this case, f must be *permutation equivariant*:

$$f(\mathbf{PAP}^\top, \mathbf{PX}) = \mathbf{P}\, f(\mathbf{A}, \mathbf{X}). \tag{5.5}$$

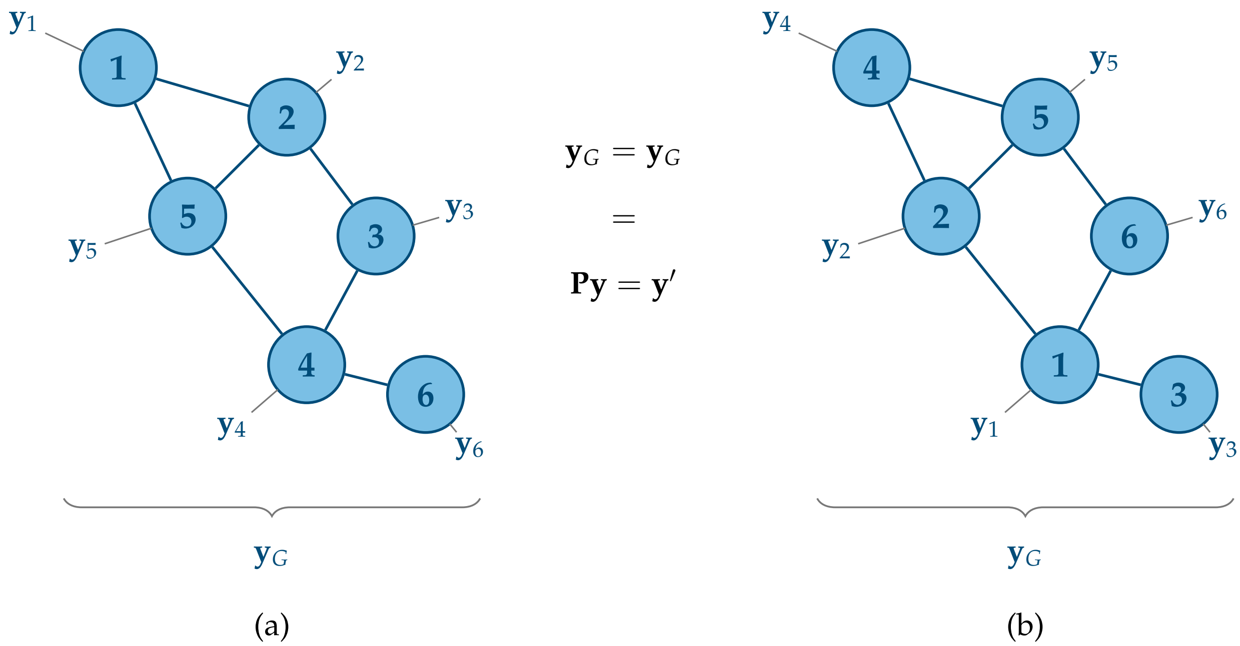


Figure 5.1: A single graph shown under two node orderings related by a permutation $\mathbf{P}$. Left: the original indexing, with a per-node output attached to each node and a single graph-level output computed from the whole graph. Right: the reordered indexing, with the graph-level output unchanged (invariance), while the per-node outputs appear in the permuted positions (equivariance).

Reordering the nodes reorders the per-node outputs in exactly the same way: the prediction for a given node is the same prediction, carried to that node's new position [3]. Figure 5.1 illustrates the two conditions on a single graph drawn under two orderings.

These two conditions, together with the relational inductive bias, determine what a layer operating on graphs must be. The relational inductive bias fixes where each node's information comes from. Permutation equivariance fixes how the layer must treat the indexing. This is the design the next chapter develops: the message-passing framework gives these requirements a concrete and general form.

[3] William L. Hamilton. *Graph Representation Learning*, volume 46 of *Synthesis Lectures on Artificial Intelligence and Machine Learning*. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

6 The Message-Passing Framework

Contents

A GNN must handle inputs of varying size, remain independent of node ordering, and incorporate graph structure into the computation. These requirements are captured by the relational inductive bias, together with the conditions of permutation invariance and equivariance. This chapter constructs a layer that satisfies these properties by design, assembling it from three component functions, examining the effects of stacking such layers, and interpreting the resulting formulation as a general design space for GNNs.

6.1 From Relational Bias to Neighborhood Aggregation

The relational inductive bias of Section 5.2 asserts that a node is characterized by its neighborhood. A layer that respects this bias therefore computes a node's representation from the information contained in the node and in its neighborhood. This is the central idea. The question is how to carry out this computation while satisfying the requirements established in the previous chapters.

The CNN offers an instructive point of comparison, because it faces the same problem on a fixed-size domain and solves it in a way that does not transfer directly [1]. A convolutional layer reads each position together with a fixed window of surrounding positions and applies the same filter at every position. Two aspects of this design are worth isolating. The window is of fixed size and fixed shape, so the filter always reads the same number of inputs arranged in the same way; and the filter is reused everywhere, so a single set of parameters serves the whole input. A graph layer should follow the same instinct — read a node together with its local region rather than the whole graph — but the neighborhood structure of a graph differs fundamentally. A neighborhood is not a fixed window: different nodes have different numbers of neighbors, so the operation must accept a variable number of inputs. And a neighborhood has no fixed arrangement: it is a set, with no first or last element, so the operation must not depend on the order in which the neighbors are listed.

[1] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

These two obstacles are met by two ingredients acting together, and the distinction between what each contributes is worth drawing precisely, since it is easy to attribute to one what belongs to the other.

The first ingredient is the *aggregation*: an operation that combines a neighborhood of arbitrary size into a single vector. This resolves the problem of variable size directly, its output has a fixed dimension regardless of how many neighbors entered it. If, in addition, that operation is insensitive to the order of its inputs, it also discards the arbitrary ordering. The aggregation, then, handles the variable size and supplies invariance to the ordering of neighbors.

The second ingredient is *parameter sharing*: the same functions, with the same parameters, are applied at every node of the graph, exactly as a single filter is reused at every position of an image. Because the identical computation is performed at every node, a relabeling of the nodes does not change the computation applied to them. Combined with an order-insensitive aggregation, it allows the desired permutation properties to be built into the layer.

Parameter sharing also keeps the number of parameters independent of the number of nodes. The same layer can therefore be applied to graphs of arbitrary size, which both improves scalability and permits a model trained on one graph to be applied to another [2]. The two ingredients are therefore complementary — the parameter sharing guarantees that the same computation is applied to every node, and the aggregation collapses a variable, unordered neighborhood into a fixed vector — and together they turn the relational inductive bias into an operation that meets the requirements of the two preceding chapters.

[2] Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7

6.2 The Message-Passing Layer

The ideas developed in the previous section can now be stated formally. A layer of the kind described takes the representations of all nodes and produces new representations for each node, each formed from the node and its neighborhood. This section assembles such a layer from three functions — one that forms a message from a neighbor, one that aggregates the messages of a neighborhood, and one that updates a node from its aggregated neighborhood — and states the general equation obtained by combining them. The unification of these three functions into a single framework, under which many previously separate graph architectures appear as instances, is due to Gilmer et al. [3]; the presentation here follows the notation and formulation used throughout this work.

[3] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, 2017. https://arxiv.org/abs/1704.01212

The object the layer transforms is the *hidden representation* of a node. Writing $\mathbf{h}_i^{(k)} \in \mathbb{R}^{d'}$ for the representation of node i after layer k, the layer maps the node representations $\{\mathbf{h}_i^{(k-1)}\}$ to updated representations $\{\mathbf{h}_i^{(k)}\}$. The representation at layer zero is initialized to the input features, $\mathbf{h}_i^{(0)} = \mathbf{x}_i$, so the first layer reads the raw node attributes and each subsequent layer reads the output of the one before. The representations produced by the intermediate layers are therefore hidden. As with the feature matrix $\mathbf{X}$, whose row i is the feature vector $\mathbf{x}_i$, the hidden representations of all nodes are collected into a matrix $\mathbf{H}^{(k)} \in \mathbb{R}^{n\times d'}$ whose row i is $\mathbf{h}_i^{(k)}$; the matrix form is used when a layer is written as a single operation on the whole graph.

6.2.1 The Message Function

The first function specifies what a neighbor contributes. For a node i and a neighbor $j \in \mathcal{N}(i)$, the *message function* $\phi^{(k)}$ produces a vector — the message sent from j to i — from the representations available at the two endpoints of the edge and, where present, the feature of the edge itself:

$$\mathbf{m}_{j\to i}^{(k)} = \phi^{(k)}\left(\mathbf{h}_i^{(k-1)}, \mathbf{h}_j^{(k-1)}, \mathbf{e}_{j,i}\right). \tag{6.1}$$

The function $\phi^{(k)}$ is a map whose output $\mathbf{m}_{j\to i}^{(k)}$ is the message; its arguments are the representation of the receiving node i, the representation of the sending neighbor j, and the feature $\mathbf{e}_{j,i}$ of the edge from j to i. Not every argument need be used: a message may be formed from the neighbor alone, from the neighbor together with the central node, or from either of these together with the edge feature. The general form admits all three sources — the neighbor, the central node, and the edge — and a particular architecture selects among them.

The resulting message therefore carries exactly the information that the message function allows to pass across the edge before neighborhood-level aggregation takes place.

6.2.2 The Aggregation Function

The message function produces one message for each neighbor of i. Since a node may have any number of neighbors, these messages form a collection of variable size, and they must be combined into a single vector before the node can be updated. The *aggregation function* $\bigoplus$ performs this combination over the neighborhood:

$$\mathbf{a}_i^{(k)} = \bigoplus_{j \in \mathcal{N}(i)} \mathbf{m}_{j \to i}^{(k)}. \tag{6.2}$$

The aggregated vector $\mathbf{a}_i^{(k)}$ summarizes the entire neighborhood of i in a single vector of fixed dimension, whatever the number of neighbors. The neighborhood $\mathcal{N}(i)$ is a set, and the messages carry no intrinsic order; the result of combining them must therefore not depend on the order in which they are taken. The aggregation function must be invariant to the ordering of its inputs. The operations commonly used — the sum, the mean, and the maximum, taken coordinatewise — all satisfy this condition, and any of them yields a well-defined aggregation.

6.2.3 The Update Function

The aggregated neighborhood does not by itself determine the node's new representation. Although information about the node itself may already enter through the message function, the layer still requires a mechanism that combines the neighborhood summary with the node's own prior representation. The *update function* $\gamma^{(k)}$ combines the two:

$$\mathbf{h}_i^{(k)} = \gamma^{(k)}\left(\mathbf{h}_i^{(k-1)}, \mathbf{a}_i^{(k)}\right). \tag{6.3}$$

The function $\gamma^{(k)}$ is a map that takes the node's own prior representation $\mathbf{h}_i^{(k-1)}$ together with the aggregated message $\mathbf{a}_i^{(k)}$ and returns the updated representation $\mathbf{h}_i^{(k)}$. Retaining the node's prior state alongside the neighborhood is what prevents the node's own information from being discarded; the balance between the two is a matter of how $\gamma^{(k)}$ is chosen, and ranges from a simple sum of the two arguments to a learnable combination, a question taken up with the design space in Section 6.6.

6.2.4 The General Equation of a Message Passing Layer

Composing the three functions gives the general form of a message-passing layer. Substituting the message function (6.1) into the aggregation (6.2), and the result into the update (6.3), the representation of node i at layer k is

$$\mathbf{h}_i^{(k)} = \gamma^{(k)}\left(\mathbf{h}_i^{(k-1)}, \bigoplus_{j\in\mathcal{N}(i)} \phi^{(k)}\left(\mathbf{h}_i^{(k-1)}, \mathbf{h}_j^{(k-1)}, \mathbf{e}_{j,i}\right)\right). \qquad (6.4)$$

This equation defines the message-passing framework (Figure 6.1), and the architectures are obtained by choosing the three functions $\phi^{(k)}$, $\bigoplus$, and $\gamma^{(k)}$. Where any of these functions applies a learnable linear transformation, the transformation is written with a weight matrix $\mathbf{W}$, and an associated bias is always understood to be present even where it is not written: the bias is frequently absorbed into the weight notation or omitted for brevity, and its omission from a particular equation never signifies its absence. Hamilton expresses the same construction with operators named AGGREGATE and UPDATE, in place of $\bigoplus$ and $\gamma^{(k)}$; the two formulations describe the same computation [4].

[4] William L. Hamilton. *Graph Representation Learning*, volume 46 of *Synthesis Lectures on Artificial Intelligence and Machine Learning*. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6

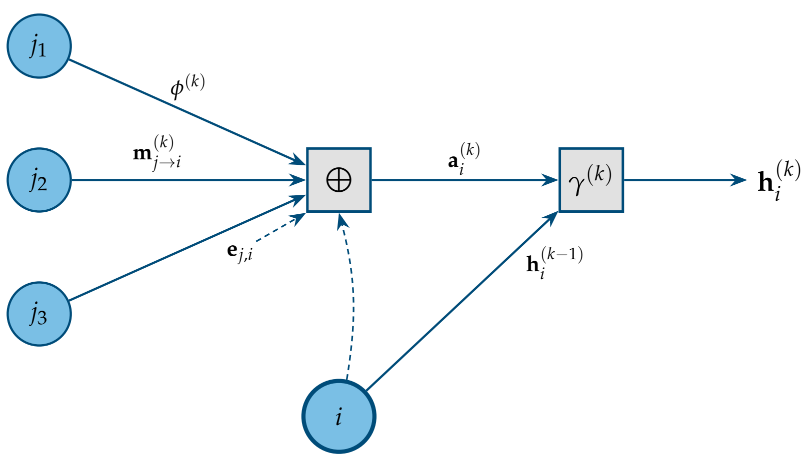


Figure 6.1: One message-passing layer applied to a central node i. For each neighbor $j \in \mathcal{N}(i)$, the message function $\phi^{(k)}$ forms a message from the representation of j and, optionally, from the representation of i and the edge feature. The aggregation $\bigoplus$ combines the neighbors' messages into a single vector, invariant to their order. The update function $\gamma^{(k)}$ combines this aggregate with the node's own previous representation to produce $\mathbf{h}_i^{(k)}$.

The layer assembled in this way meets the permutation conditions of Section 5.4: the message function (6.1) is applied identically to every edge, the aggregation (6.2) is invariant to the order of the neighbors, and the update (6.3) is applied identically to every node. Relabeling the nodes by a permutation therefore does not change any node's computed representation; it only moves that representation to the node's new index. In the matrix notation of Section 5.4, a layer that sends $(\mathbf{A}, \mathbf{H}^{(k-1)})$ to $\mathbf{H}^{(k)}$ sends the relabeled graph $(\mathbf{PAP}^\top, \mathbf{PH}^{(k-1)})$ to $\mathbf{PH}^{(k)}$, which is the condition of permutation equivariance in (5.5). The

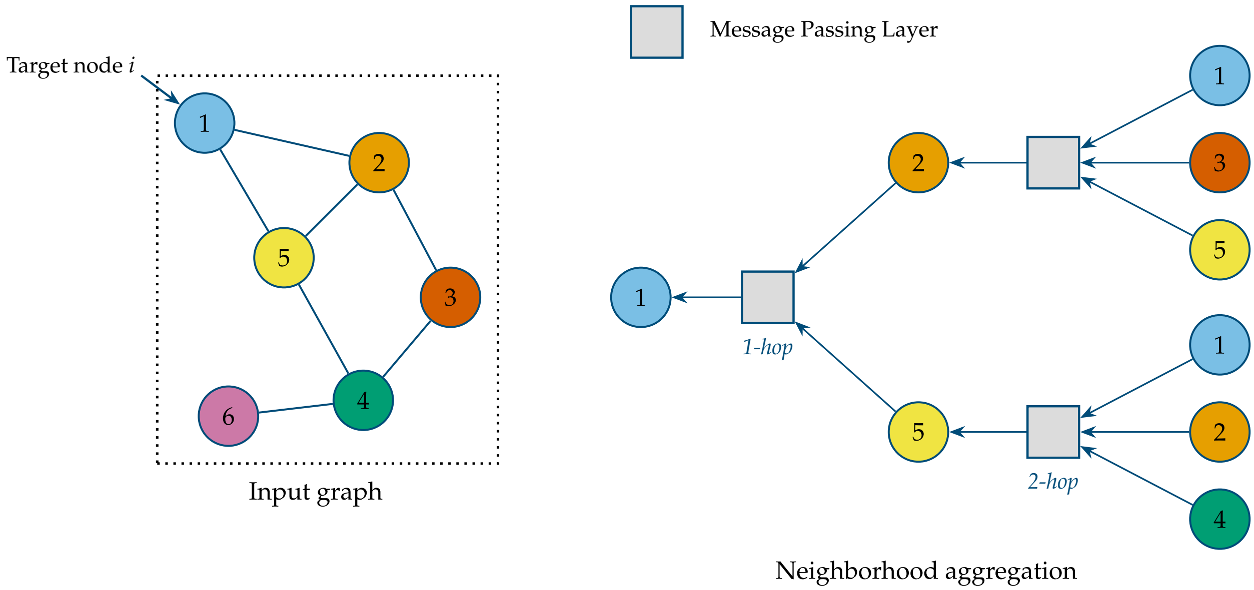


Figure 6.2: Left: a target node in the input graph. Right: the computation that produces its representation, unrolled over message-passing layers. The first layer aggregates messages from the node's immediate neighbors; the second aggregates messages from the neighbors of those neighbors, and so on. After k layers the representation of the target node depends on every node within k hops of it, the receptive field at depth k.

per-node output follows the nodes when they are reordered, as a node-level task requires. A task defined on the whole graph requires instead a single output unchanged by relabeling — the permutation invariance of (5.4) — and this is obtained by reducing the per-node representations to one vector with a further order-insensitive operation; that operation, called *readout*, is the one used by graph-level architectures, and is developed in Chapter 7.

6.3 Receptive Field and Depth

A single message-passing layer updates each node from its immediate neighbors. Applying the layer once therefore lets each node incorporate information from one step away. To reach further, layers are stacked: the output representations of one layer become the input representations of the next, and a network of K layers applies the update (6.4) K times in succession, with $k = 1, \dots, K$. Each layer carries its own functions $\phi^{(k)}, \gamma^{(k)}$ and its own parameters; the parameters are shared across the nodes within a layer, as Section 6.1 established, but not across layers, so the transformation applied at depth k differs from the one applied at depth $k-1$.

Stacking has a definite effect on how far information can propagate into a node's representation, and the effect compounds with depth. After one layer, $\mathbf{h}_i^{(1)}$ depends on the nodes in $\mathcal{N}(i)$, the neighbors at distance one. After two layers, $\mathbf{h}_i^{(2)}$ is computed from the first-layer representations of those neighbors, and each of those already depends on its own neighbors; the representation of i thus comes to depend on the nodes within distance two. In general, after k layers the representation $\mathbf{h}_i^{(k)}$ depends on every node within geodesic distance k of i — the set of nodes j with $d(i, j) \leq k$, the distance being the one defined in Chapter 3. This is the *receptive field* of node i at depth k: the region of the graph that can influence the node's representation. Each additional layer expands the receptive field by one *hop*, so the depth of the network sets the radius of the neighborhood that each node ultimately incorporates (Figure 6.2).

The depth of the network thus controls the extent of the graph that each node's representation can incorporate, and this is the parameter a designer sets when choosing the number of layers. The final representations $\mathbf{h}_i^{(K)}$, produced after the last of the K layers, are the *embeddings* of the nodes: the learned vectors that summarize each node together with its K-hop context, and that serve as the input to the task-specific prediction function.

6.4 Transductive and Inductive Learning

The way a message-passing network is trained and evaluated depends on what portion of the graph data is available during training. In practice, the node- and graph-level tasks introduced in Chapter 2 commonly give rise to two distinct learning settings. The distinction follows directly from the fact that a node's representation is computed from its neighborhood.

Consider first a task defined at the node level, in which each node carries a label and the goal is to predict the labels of some nodes from those of others. As in any supervised problem, the nodes are partitioned into a training set, whose labels are used to fit the model, and an evaluation set, whose labels are withheld and used to measure generalization. The graph structure, however, prevents these two sets from being separated cleanly. When the network computes the representation of a training node, it aggregates the representations of that node's neighbors, and some of those neighbors belong to the evaluation set. Their features and their position in the graph therefore enter the computation performed during training, even though their labels do not. This is the *transductive* setting. In node-classification problems it is often described as a form of *semi-supervised learning*, since the labels of only a subset of the nodes are available during training.

A task defined at the graph level admits a cleaner separation. Here the dataset is a collection of whole graphs, each carrying a single label, and the partition into training and evaluation sets is a partition of the graphs. A graph held out for evaluation is absent from training in its entirety — its nodes, its edges, and its label are all unseen — because the unit of the partition is the whole graph rather than a node within a shared structure. This is the *inductive* setting, corresponding to the usual notion of training on some examples and generalizing to others never seen before.

6.5 *Expressive Power and the 1-WL Bound*

The framework constructed so far guarantees that a message-passing layer respects the permutation conditions of a graph, but it says nothing about how much the resulting representations can distinguish. A natural way to measure the capability of a graph model is to ask which graphs it can tell apart: given two graphs that are not isomorphic, does the network assign them different representations, or does it map structurally different graphs to the same output? This is the question of *expressive power*, and it is the property that governs how well a model can serve graph-level tasks, where distinguishing one graph from another is the whole of what is required. Graph isomorphism, defined in Chapter 3, is the precise notion of two graphs being the same up to a relabeling; expressive power asks how close a network comes to distinguishing every pair of graphs that are genuinely different.

There is a classical procedure for distinguishing non-isomorphic graphs that was introduced before GNNs and turns out to mirror message passing closely. The one-dimensional Weisfeiler–Leman test, abbreviated 1-WL, assigns a color to every node and refines these colors iteratively. Initially all nodes share a common color. At each round, a node's new color is determined by its current color together with the collection of colors of its neighbors; nodes that have the same color and whose neighbors have matching collections of colors keep a common color, while any difference in the surrounding colors splits them apart. The refinement continues until the partition of nodes into colors no longer changes. Two graphs for which the refinement process produces different colorings are certainly not isomorphic.

The correspondence with message passing is immediate. A round of 1-WL updates a node's color from its own color and the colors of its neighbors; a message-passing layer updates a node's representation from its own representation and the aggregated representations of its neighbors. The color plays the role of the hidden representation, and the refinement step plays the role of one layer [5]. This parallel bounds the framework from above: a message-passing network can distinguish two graphs only if 1-WL assigns them different colorings. No network built from the general equation (6.4) can separate a pair of graphs that 1-WL fails to separate. The expressive power of message passing is thus bounded above by 1-WL.

[5] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *Proceedings of the 7th International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1810.00826

This bound is a ceiling: a particular architecture may fall short of it, depending on how its aggregation and update functions are chosen. Reaching the ceiling — matching the full discriminative power of 1-WL rather than settling below it — requires the aggregation to preserve enough information about the neighborhood that distinct neighborhoods always yield distinct aggregated vectors, and motivates the architecture developed for graph-level tasks in Chapter 7. The question of what the 1-WL ceiling excludes — the structurally different graphs that no message-passing network can tell apart — is a limitation of the framework and is taken up in Chapter 9.

6.6 *The Design Space*

The equation (6.4) is not a single model but a template. It fixes a layer — a message formed per neighbor, an aggregation over the neighborhood, an update of the node — and requires only that the three functions respect the permutation conditions; it does not fix what those functions are. Every choice of $\phi^{(k)}$, $\bigoplus$, and $\gamma^{(k)}$ that meets the conditions produces a valid layer, and the space of such choices is the design space of the framework.

6.6.1 *Flexibility of the Framework*

The flexibility of the equation can be seen function by function. The message function $\phi^{(k)}$ may be as simple as a linear transformation of the neighbor's representation, or as involved as an MLP applied to the neighbor, the central node, and the edge feature together. The aggregation $\bigoplus$ may be any order-insensitive operation, with the sum, mean, and maximum being the common choices, each preserving a different aspect of the neighborhood. The update $\gamma^{(k)}$ may be a linear map followed by a nonlinearity, an MLP, or a rule that concatenates the node's prior representation with the aggregated neighborhood, $\mathbf{h}_i^{(k-1)} \oplus \mathbf{a}_i^{(k)}$, before transforming the result, so that the two are kept distinct. Each such decision is a coordinate in the design space, and a complete architecture is a setting of all of them.

The flexibility extends beyond the choice of functions to the very object whose representation is learned. The equation forms a node's representation from messages along its edges, but nothing in its structure requires the unit of representation to be a node. A layer that built its messages from edge features and aggregated over adjacent edges would, by the same mechanism, learn representations of the edges themselves; the message, aggregation, and update functions retain their roles, and only the object they act on changes. Edge-centered architectures found in the literature are instances of the same framework, reoriented from nodes to edges. This generality indicates that message passing is not tied to node representations specifically, but is a template for computing on graph elements through local aggregation.

6.6.2 The MLP as a Degenerate Case

One point in the design space recovers a familiar model and locates the framework relative to it. Consider a graph with no edges — a set of nodes carrying no relations, so that each node is independent. There, the neighborhood $\mathcal{N}(i)$ is empty for every node, so the message function has nothing to act on and the aggregation contributes nothing; the update reduces to $\mathbf{h}_i^{(k)} = \gamma^{(k)}(\mathbf{h}_i^{(k-1)})$, and each node is transformed independently of the rest. When $\gamma^{(k)}$ is itself an MLP, the layer becomes an MLP applied separately to every node — precisely the model that Chapter 4 showed to be unable to incorporate graph structure, now recovered as the case in which there is no structure to incorporate.

Read in the other direction, this shows that a message-passing network is a generalization of the MLP, with the MLP appearing as a particular instance of the framework.

6.6.3 Established Architectures as Instances

The design space is not merely a catalog of possibilities; it is populated by architectures that have been proposed, studied, and applied, each fixing the three functions in a particular way to serve a particular purpose. The recognition that these architectures, developed separately, are instances of one framework is itself the contribution of the message-passing formulation [6]: models that appear distinct on the surface differ only in their choice of message, aggregation, and update.

This work examines three such instances, one representative of each task level introduced in Chapter 2: a node-level architecture built on the assumption that adjacent nodes are similar; an edge-level architecture that uses node embeddings to score the plausibility of a link between two nodes; and a graph-level architecture whose aggregation is chosen to reach the 1-WL ceiling of Section 6.5, maximizing the network's power to distinguish whole graphs. Each is the subject of the following chapter, where the general framework of this chapter is given a concrete form.

[6] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, 2017. https://arxiv.org/abs/1704.01212

7 Representative Architectures for Node-, Edge-, and Graph-Level Tasks

Contents

The message-passing framework of Chapter 6 fixes the form of a GNN layer without fixing its content: a message per neighbor, an order-insensitive aggregation, and an update, with the three functions left open. The design space of Section 6.6 consists of the admissible choices for these functions. This chapter selects three points in that space, one for each of the task levels introduced in Chapter 2. Each architecture is presented as an instance of the general equation (6.4), and each is paired with the elements its task demands beyond the message-passing layer itself: how a per-node prediction is formed for node-level tasks, how a pair of nodes is scored for edge-level tasks, and how the per-node representations are reduced to a single vector for graph-level tasks.

7.1 Node-Level Learning

A node-level task assigns a prediction to each node of a single graph. Chapter 6 established that stacking K layers produces, for every node, an embedding $\mathbf{h}_i^{(K)}$ that summarizes the node together with its K-hop context, and that the per-node output of an equivariant network follows the nodes under relabeling, as a node-level task requires. Chapter 6 also established the transductive setting in which such a task is trained. What remains is to specify how a prediction is computed from each embedding, and then to fix the three functions of the layer.

7.1.1 From Embeddings to Predictions

In node classification each node carries a label drawn from a fixed set of classes, and the goal is to predict the labels of the unlabeled nodes. The embedding $\mathbf{h}_i^{(K)}$ produced by the final message-passing layer is the representation on which this prediction is based. A prediction is obtained by applying a *classifier* to each node's embedding: a function shared across nodes that produces a prediction from each embedding. A common choice is a linear classifier, which maps the embedding to one score per class through a learnable linear transformation and then normalizes these scores to a probability distribution,

$$\hat{\mathbf{y}}_i = \mathrm{softmax}\left(\mathbf{W}\,\mathbf{h}_i^{(K)}\right), \tag{7.1}$$

where $\mathbf{W}$ is a weight matrix with one row per class, the bias understood as in Section 6.2.4, and softmax the function that turns a vector of scores into nonnegative entries summing to one. The entry of $\hat{\mathbf{y}}_i$ with largest value is the predicted class of node i. Because the classifier is shared across nodes and the embeddings are produced equivariantly, the prediction attached to a node is unchanged by a relabeling of the graph, that is, it moves with the node.

7.1.2 The Graph Convolutional Network

The *Graph Convolutional Network (GCN)* is the instance of the message-passing framework obtained by taking the aggregation to be a normalized sum over the closed neighborhood, each term weighted by the degrees of its two incident nodes [1]. It is presented here first in the per-node form that exhibits it as an instance of the general equation, and then in the matrix form that connects it to the spectral perspective of Section 3.4.

[1] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017. https://arxiv.org/abs/1609.02907

As an instance of the general message-passing equation, GCN fixes its three functions as follows. Messages are aggregated over the closed neighborhood $\mathcal{N}(i) \cup \{i\}$ by summing the representations of the node and its neighbors, each weighted by a normalization coefficient, so that the aggregation is a normalized sum. The update then applies a shared linear transformation followed by an elementwise nonlinearity. The resulting layer computes

$$\mathbf{h}_i^{(k)} = \sigma\left(\mathbf{W}^{(k)} \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\widetilde{\deg}(i)}\,\sqrt{\widetilde{\deg}(j)}}\,\mathbf{h}_j^{(k-1)}\right), \tag{7.2}$$

where $\mathbf{W}^{(k)}$ is the shared weight matrix, σ is an elementwise nonlinearity, and $\widetilde{\deg}(v) = \deg(v) + 1$ denotes the degree of any node v after adding a self-loop.

Every term of the sum — including the node's own, entering through the self-loop with $j = i$ — is weighted by the coefficient $1/\sqrt{\widetilde{\deg}(i)\widetilde{\deg}(j)}$. This *symmetric normalization* scales each contribution by the degrees of the two incident nodes, preventing the aggregated representation from growing with the size of the neighborhood and reducing the influence of highly connected nodes.

Because the sum runs over the closed neighborhood rather than over $\mathcal{N}(i)$ alone, the separate update of the central node used in the general framework is not performed as a distinct step. The node and its neighbors are folded into a single normalized aggregation and processed by the same linear transformation.

Applying the same operation to all nodes yields the matrix form of the layer. The self-loops and the degree normalization are captured by two matrices. Adding a self-loop at every node gives

$$\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}, \qquad \tilde{D}_{ii} = \sum_j \tilde{A}_{ij}, \tag{7.3}$$

where $\tilde{\mathbf{A}}$ is the adjacency matrix with self-loops and $\tilde{\mathbf{D}}$ is its degree matrix, the diagonal matrix holding the row sums of $\tilde{\mathbf{A}}$. With these matrices the layer is

$$\mathbf{H}^{(k)} = \sigma\Big(\tilde{\mathbf{D}}^{-1/2}\,\tilde{\mathbf{A}}\,\tilde{\mathbf{D}}^{-1/2}\,\mathbf{H}^{(k-1)}\,\mathbf{W}^{(k)}\Big)\,, \tag{7.4}$$

where $\mathbf{H}^{(k)}$ is the matrix of node representations of Section 6.2. The matrix $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ that acts on $\mathbf{H}^{(k-1)}$ is the *propagation operator* of the layer, the object that mixes each node's representation with those of its neighbors. Its factors $\tilde{\mathbf{D}}^{-1/2}$ on each side of $\tilde{\mathbf{A}}$ supply the symmetric normalization, reproducing the coefficient $1/\sqrt{\widetilde{\deg}(i)}\sqrt{\widetilde{\deg}(j)}$ of the per-node form for each pair.

The propagation operator connects the layer to the spectral perspective of Section 3.4. In the original formulation, GCN is obtained by applying a first-order approximation to a spectral graph convolution, yielding the operator $\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ acting on the node representations before the learnable linear transformation. In terms of the symmetric normalized Laplacian of Equation (3.13), this operator equals $2\mathbf{I} - \mathbf{L}_{\text{sym}}$. This is the sense in which the layer is *spectral*: it acts through a fixed function of the graph Laplacian, the operator whose spectrum Section 3.4 showed to be tied to the connectivity of the graph. The intermediate steps of this spectral derivation are beyond the present scope, which uses only the resulting operator; the complete derivation is given in the original work [2].

[2] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017. https://arxiv.org/abs/1609.02907

The renormalization trick (7.3) is what turns this first-order operator into the propagation operator the layer actually applies. It addresses a spectral difficulty: the eigenvalues of $\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ lie in the interval $[0, 2]$, so applying it repeatedly, as a deep stack of layers does, can amplify the node representations to the point of numerical instability [3]. Substituting $\tilde{\mathbf{A}}$ for $\mathbf{A}$ and $\tilde{\mathbf{D}}$ for the degree matrix of $\mathbf{A}$ shrinks this spectrum, reducing the largest eigenvalue to well below 2 in practice, which stabilizes the repeated application [4].

[3] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017. https://arxiv.org/abs/1609.02907

[4] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. https://arxiv.org/abs/1902.07153

The spectral reading also explains the effect a GCN layer has on the representations. The quadratic form $\mathbf{f}^\top \mathbf{L}\mathbf{f}$ of Equation (3.11) measures how much an assignment of values to nodes varies across edges, and the same notion of variation applies to $\mathbf{L}_{\text{sym}}$: an eigenvector of $\mathbf{L}_{\text{sym}}$ with a large eigenvalue is one whose values vary most across edges, and one with a small eigenvalue varies little. The propagation operator applies a fixed function of this Laplacian that inverts the order of its spectrum, sending the large eigenvalues to small factors and the small eigenvalues to large ones, so that the components varying most across edges are attenuated and those varying least are preserved. It thus acts as a low-pass filter on the graph [5], and repeated application reduces the variation across edges, bringing the representations of adjacent nodes closer together with every layer. This is *smoothing*: a GCN layer smooths the node representations over the edges of the graph.

[5] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. https://arxiv.org/abs/1902.07153

Smoothing is the operative bias of GCN, and it is the right bias exactly when the graph is homophilous. Section 3.3 described a graph as homophilous when adjacent nodes tend to share attributes. On such a graph, making the representations of adjacent nodes more similar aligns the learned representations with the labels, which is why GCN is effective for node classification. The same mechanism becomes a liability when the assumption fails — when adjacent nodes differ systematically, or when so many layers are stacked that all representations are smoothed toward a common value — a limitation taken up in Chapter 9.

7.2 Edge-Level Learning

An edge-level task makes predictions about pairs of nodes rather than nodes individually. The representative task, introduced in Chapter 2, is link prediction: given the edges observed in a graph, predict which further edges are likely to exist. A common approach to this setting is provided by the encoder–decoder framework, and requires two elements absent from the node-level case: a way to score a pair of nodes from their embeddings, and a way to construct a supervised training signal from a graph whose only data is the presence or absence of edges.

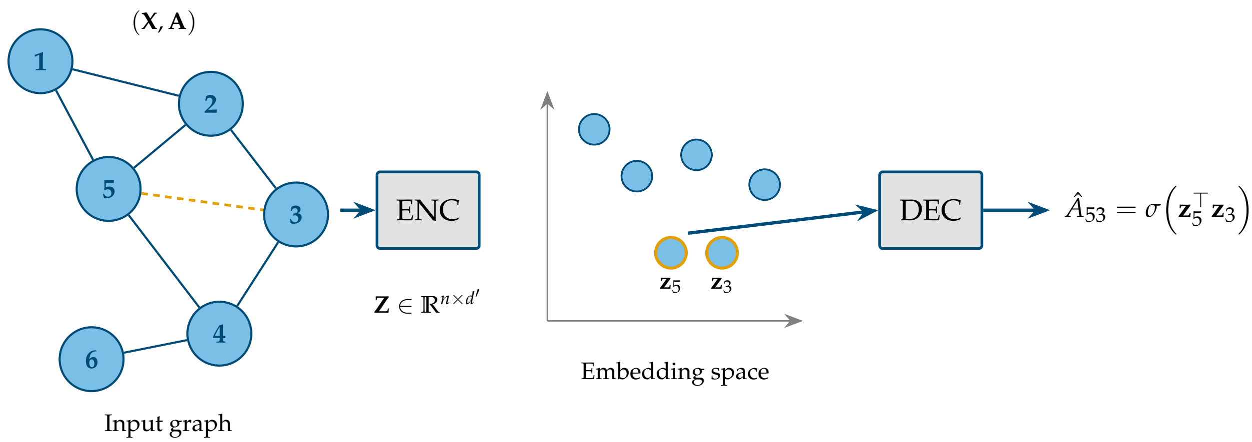


Figure 7.1: The encoder–decoder framework for link prediction. The encoder, a message-passing network, maps the graph $(\mathbf{X}, \mathbf{A})$ to node embeddings $\mathbf{Z}$, one vector $\mathbf{z}_i$ per node. The decoder takes a pair of embeddings $\mathbf{z}_i, \mathbf{z}_j$ and returns a scalar score $\hat{A}_{ij} = \sigma(\mathbf{z}_i^\top \mathbf{z}_j)$, the predicted probability of an edge between i and j. A high score is assigned where the encoder has placed the two embeddings close together.

7.2.1 *The Encoder–Decoder Framework*

The encoder-decoder framework divides a link-prediction model into two stages (Figure 7.1). The *encoder* is a message-passing network of the kind built in Chapter 6: it maps the graph to node embeddings,

$$\mathbf{Z} = \mathrm{ENC}(\mathbf{X}, \mathbf{A}), \tag{7.5}$$

where $\mathbf{Z} \in \mathbb{R}^{n \times d'}$ collects the final embeddings, its row i being the embedding $\mathbf{z}_i = \mathbf{h}_i^{(K)}$ of node i. The encoder carries all the graph structure into the embeddings; what remains is to turn a pair of embeddings into a prediction about the edge between them. The *decoder* performs this second step. It is a function that maps a pair of embeddings to a scalar score expressing how plausible an edge between the two nodes is. A suitable choice is the *inner product decoder* [6], which scores a pair by the inner product of their embeddings passed through a function that maps it to a probability,

$$\hat{A}_{ij} = \sigma\left(\mathbf{z}_i^\top \mathbf{z}_j\right), \tag{7.6}$$

where σ is the logistic function, mapping the real-valued score to a value in $(0, 1)$ read as the predicted probability of an edge between i and j. The inner product is large when the two embeddings point in a similar direction, so the decoder assigns high probability to pairs whose embeddings the encoder has placed close together. The decoder requires no parameters of its own, which places the full responsibility for the prediction on the embeddings.

The encoder must therefore arrange the embeddings so that connected nodes are close and unconnected ones far apart. To train it toward this arrangement, a supervised signal is needed, and the structure of the graph itself supplies it. The edges actually present in

[6] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. In *NIPS Workshop on Bayesian Deep Learning*, 2016. https://arxiv.org/abs/1611.07308

the graph are the *positive examples*: pairs that should receive a high score. Pairs of nodes with no observed edge between them are the *negative examples*: pairs that should receive a low score. A link-prediction model is trained to raise the score on positive pairs and lower it on negative pairs, exactly as a binary classifier is trained to separate two classes.

Two practical points govern how these examples are formed. First, the positive edges are split, as in any supervised problem: a portion is held out for evaluation and removed from **A** during training, not merely from the loss, since otherwise the encoder would see the very edges it is meant to predict. Second, the negative examples must be sampled, because non-edges vastly outnumber observed edges: a graph has at most $\binom{n}{2} = n(n-1)/2$ possible edges but typically far fewer actual ones. The standard remedy is *negative sampling*, which draws a random subset of non-edges, comparable in number to the positive edges, so that the training signal remains balanced.

7.2.2 The Graph Autoencoder

The *Graph Autoencoder (GAE)* is the instance of the encoder–decoder framework that pairs a GCN encoder with the inner product decoder [7]. Its name comes from the autoencoder pattern: a model that encodes its input into a latent representation and then reconstructs the input from that representation. Here the input reconstructed is the adjacency matrix. The encoder is a GCN, producing the embedding matrix **Z** as in (7.5); the decoder applies the inner product (7.6) to every pair of embeddings at once, reconstructing the full adjacency matrix as

$$\hat{\mathbf{A}} = \sigma\left(\mathbf{Z}\mathbf{Z}^\top\right), \tag{7.7}$$

where σ is applied elementwise and the entry $\hat{A}_{ij}$ is the predicted probability of an edge between i and j.

[7] Thomas N. Kipf and Max Welling. Variational graph auto-encoders. In *NIPS Workshop on Bayesian Deep Learning*, 2016. https://arxiv.org/abs/1611.07308

Training matches the reconstructed adjacency against the observed one on the positive and sampled negative pairs: the GCN encoder is fitted so that the embeddings it produces, when combined by the inner product, reproduce the edges of the graph. The arrangement the encoder learns is the one the decoder rewards — embeddings of connected nodes made similar, those of unconnected nodes made dissimilar — which is the arrangement a GCN produces by construction, since its smoothing brings the embeddings of adjacent nodes together.

7.3 Graph-Level Learning

A graph-level task assigns a single prediction to an entire graph. The representative task, introduced in Chapter 2, is graph classification: assigning a label to a whole graph, as in predicting a property of a molecule. This setting requires an element not needed in the previous levels — a way to reduce the per-node embeddings, which vary in number from graph to graph, to a single fixed-size vector representing the graph — and it places a demand on the message-passing layer that the previous tasks did not, since graph classification requires the layer to preserve the structural information that distinguishes one graph from another.

7.3.1 Pooling and Readout

A message-passing network produces one embedding per node. A graph-level prediction requires one vector for the whole graph, so the per-node embeddings must be combined into a single graph embedding. This reduction is the *readout* operation named in Section 6.2.4, also called *global pooling*: a function that maps the collection of node embeddings to one vector,

$$\mathbf{h}_{\mathcal{G}} = \mathrm{READOUT}\Big(\{\, \mathbf{h}_i^{(K)} : i \in V \,\}\Big)\,, \tag{7.8}$$

where $\mathbf{h}_{\mathcal{G}}$ is the graph embedding, on which a classifier of the form (7.1) is then applied to predict the graph's label.

The readout is subject to one essential requirement: a graph-level prediction must be unchanged by a relabeling of the nodes, so the readout must be invariant to the order of its inputs. It takes a collection of node embeddings and must return the same vector whatever order they are listed in. This is the permutation invariance of Equation (5.4), and it restricts the readout to the order-insensitive operations of Section 6.2.2: the sum, the mean, and the maximum, taken coordinatewise over the node embeddings. The readout is neighborhood aggregation applied once more, over all nodes at once.

The choice among sum, mean, and maximum is not neutral. The mean and the maximum can discard information that distinguishes graphs: the mean of a set is unchanged when the set is duplicated, and the maximum ignores all but the extreme value in each coordinate, so two graphs differing in the multiplicity or the distribution of their node embeddings can be mapped to the same vector. The sum retains more, because it grows with the number of nodes and reflects the full collection rather than its average or its extremes. This difference is decisive for the architecture that follows, whose design begins from the requirement that no distinguishing information be discarded.

7.3.2 The Graph Isomorphism Network

The *Graph Isomorphism Network (GIN)* is the instance of the message-passing framework designed to retain as much distinguishing information as the framework allows [8]. Its motivation is the expressive-power question introduced in Section 6.5: a graph-level model is only as good as its ability to tell non-isomorphic graphs apart, and, as established there, no message-passing network can exceed the discriminative power of the 1-WL test. GIN is the architecture that reaches this ceiling, and the requirement that fixes its design is the preservation of neighborhood information through the aggregation.

[8] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *Proceedings of the 7th International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1810.00826

The requirement can be stated precisely with one notion not yet needed. A neighborhood, taken as the collection of its neighbors' representations, may contain the same representation more than once, since distinct neighbors may carry identical vectors. The right object is therefore not a set, which records only which elements are present, but a *multiset*, a collection in which elements may repeat and the number of repetitions is part of the object. For the aggregation to preserve all the information in a neighborhood, it must be *injective* over multisets: it must map distinct neighborhood multisets to distinct vectors, never collapsing two different neighborhoods to the same aggregate. The mean and the maximum fail this requirement, for the reasons discussed previously. The sum, by contrast, can be made injective over multisets, because it reflects every element and its multiplicity.

GIN builds its layer on the sum accordingly. The aggregation is the sum over the neighborhood; the update combines the node's own representation with this sum and passes the result through a function expressive enough to preserve the distinction, giving

$$\mathbf{h}_i^{(k)} = \mathrm{MLP}^{(k)}\left(\left(1+\epsilon^{(k)}\right)\mathbf{h}_i^{(k-1)} + \sum_{j\in\mathcal{N}(i)} \mathbf{h}_j^{(k-1)}\right), \tag{7.9}$$

where $\mathrm{MLP}^{(k)}$ is the update function of the layer, and $\epsilon^{(k)}$ is a scalar, which can be fixed or learnable. The MLP in the update can approximate an injective function of its input, so the composition of the sum and the update preserves the distinction between different closed neighborhoods. The term $(1+\epsilon^{(k)})$ keeps the node's own representation distinguishable from the aggregate of its neighbors, rather than letting the two merge as they do in the closed-neighborhood sum of GCN. Together these choices make one GIN layer as discriminating as one round of the 1-WL refinement of Section 6.5, which is the most a message-passing layer can be.

8 Implementations and Experimental Evaluation

Contents

The architectures of Chapter 7 are now brought to experimental evaluation. Each is implemented on a standard benchmark and evaluated against an MLP baseline that ignores the structure of the graph, so that whatever performance separates them is attributable to the relational inductive bias of Section 5.2, under which the examples are related rather than independent. This comparison directly addresses the objective stated in Section 1.2. The chapter first establishes the methodology common to all three implementations and then presents each in turn: GCN on the Cora benchmark dataset for node classification, GAE on Amazon Computers for link prediction, and GIN on NCI1 for graph classification.

8.1 Experimental Methodology

All three implementations are built in PyTorch Geometric [1], a library that provides graph data structures, benchmark loaders, and message-passing layers, and all three follow one experimental pipeline. A

[1] Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. https://arxiv.org/abs/1903.02428

benchmark dataset is loaded and prepared; the GNN and the MLP are instantiated; both are trained under identical optimization settings, with a validation set monitored to select the best model; and the selected models are evaluated once on a held-out test set.

The datasets are established benchmarks available through PyTorch Geometric. For this reason, no exploratory analysis or additional cleaning is performed, and preparation is limited to the transformations and splits described in the corresponding section.

No hyperparameter search is performed. Instead, the architectures and optimization settings are selected to provide stable learning, seeking configurations that avoid underfitting and overfitting as far as each architecture permits. The goal is to ensure that neither the GNNs nor the MLPs are disadvantaged by poor configuration, so that the comparison isolates the effect of incorporating graph structure through message passing.

Reproducibility is enforced through controlled execution conditions. All experiments are executed on CPU, avoiding hardware-dependent nondeterminism introduced by parallel GPU operations. Under this setting, every stochastic component of a run — the initialization of parameters, the sampling of splits, the drawing of negative examples, and other operations that consume randomness — is governed by a single seed, so that a run is reproducible once its seed is fixed. Each experiment is repeated over a fixed set of ten seeds. Reporting performance over multiple seeds provides a more robust assessment than a single run by accounting for the variability induced by stochastic training. For each model, the evaluation metric is summarized by its median, mean, and standard deviation across the ten runs. The figures show the run corresponding to the median performance, ensuring that every figure corresponds to an actual run rather than to an aggregate that does not correspond to any individual run.

The companion repository [2] contains the implementation required to reproduce this chapter: the configurations, the dataset loading and preparation, the training and evaluation pipeline, the scripts that generate every figure and compute every metric, and the exact dependency manifest.

[2] Rodrigo Emmanuel Macias Pantoja. When GNNs Beat MLPs. GitHub repository, 2026. URL https://github.com/roymacias/when-gnns-beat-mlps

8.2 *Node Classification with GCN*

The first implementation addresses node classification. GCN (Section 7.1.2) and an MLP are trained to predict the topic of each publication in the Cora citation network.

8.2.1 The Cora Dataset

Cora is a citation network of machine-learning publications [3]. Nodes represent papers, each edge represents an undirected citation between two papers, each node carries a binary bag-of-words vector indicating which words from a fixed dictionary occur in the paper, and each node is labeled with one of seven topics. Although citations are inherently directed, the benchmark represents them as undirected edges to allow the information to be propagated in both directions during message passing. Table 8.1 summarizes the dataset.

[3] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 29(3):93–106, 2008. doi: 10.1609/aimag.v29i3.2157

Property	Value
Nodes	2,708
Edges	5,278
Average degree	3.9
Node feature dimension	1,433
Classes	7
Split (train/val/test nodes)	140 / 500 / 1,000

Table 8.1: Summary of the Cora benchmark dataset.

The dataset is provided as a benchmark through PyTorch Geometric, so no additional cleaning is performed. The only transformation applied to the node features is a row-wise L1 normalization.

The predefined data split is adopted without modification. The dataset ships with a partition of 140 training nodes (balanced with twenty per class), 500 validation nodes, and 1,000 test nodes. The training partition therefore contains only about five percent of the nodes in the graph. The remaining 1,068 nodes belong to none of the three partitions but remain part of the graph, contributing structural information during message passing without belonging to any supervised split. This corresponds to the transductive setting described in Section 6.4, where training has access to the full graph structure and all node features, while supervision is provided only through the labeled training nodes.

8.2.2 Model and Training

Table 8.2 specifies both models. The GCN stacks two layers of (7.4), the first serving as the hidden layer and the second as the output layer, giving every node a two-hop receptive field (Section 6.3). The MLP consists of a single hidden layer of width 32 followed by the output layer, resulting in approximately twice as many trainable parameters as the GCN — wider MLP configurations overfit the 140 training labels more severely.

Table 8.3 specifies the optimization settings, identical for both models. Both produce class probabilities through a softmax transformation and

Component	GCN	MLP
Layers	2 GCN layers	2 MLP layers
Hidden dimension	16	32
Hidden activation	ReLU	
Output dimension	7	
Trainable parameters	23,063	46,119

Table 8.2: Model configuration for node classification on Cora.

are trained using cross-entropy loss and the same regularization, so the comparison isolates the effect of incorporating graph structure through message passing. Dropout [4] and L2 regularization (weight decay) are used for regularization. Training is performed in full-batch mode, where the entire graph is processed at every optimization step, as permitted by the transductive setting.

[4] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15 (56):1929–1958, 2014. https://jmlr.org/papers/v15/srivastava14a.html

Parameter	Value
Loss	Cross-entropy
Optimizer	Adam
Learning rate	0.01
Weight decay	5×10^{-4}
Dropout	0.5
Epochs	200
Batch size	full graph

Table 8.3: Training configuration for node classification on Cora, identical for both models.

The validation set is evaluated at every epoch, and the parameters achieving the highest validation accuracy are retained. The results reported in the next subsection are obtained from the selected checkpoint evaluated on the test nodes.

8.2.3 Results

Figure 8.1 shows the loss over training. Both models reduce the training loss, but the MLP reduces it faster and to a lower value. On the validation set the two diverge. The GCN validation loss continues its decreasing trend throughout training, whereas the MLP validation loss settles early while its training loss continues to decrease, suggesting overfitting. By incorporating information from neighboring nodes through message passing, the GCN exhibits substantially less separation between training and validation loss.

Performance is measured by accuracy over the held-out nodes. Accuracy provides a global measure of classification effectiveness, while the learned embeddings are examined separately to assess whether the representations organize nodes according to their classes. Table 8.4 reports the results.

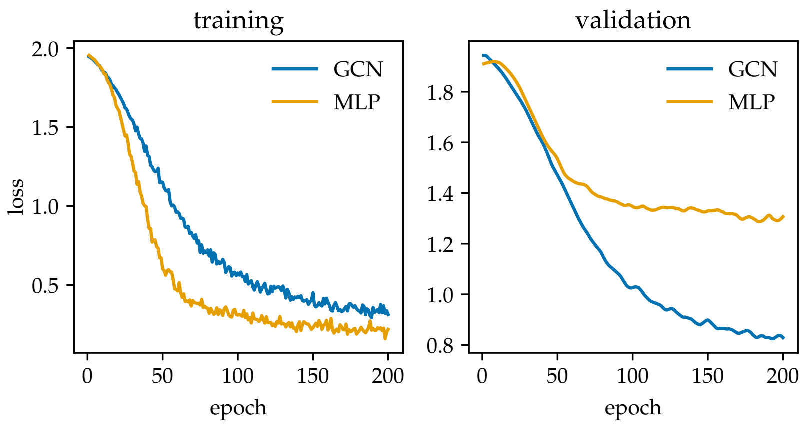


Figure 8.1: Training (left) and validation (right) loss per epoch on Cora for GCN and the MLP. Each panel is scaled to its own data.

Model	Median	Mean	Std
GCN	0.8170	0.8154	0.0061
MLP	0.5920	0.5876	0.0119

Table 8.4: Test accuracy on Cora, over ten seeds.

The GCN reaches a median accuracy of 0.8170, compared with 0.5920 for the MLP, an improvement of approximately 22.5 percentage points despite using about half as many trainable parameters. The MLP relies exclusively on the node features, whereas the GCN enriches those representations through message passing. This result is consistent with the smoothing mechanism of GCNs described in Section 7.1.2, which is effective when neighboring nodes tend to share the same class.

Figure 8.2 illustrates this difference. It projects the learned test-node representations to two dimensions using UMAP [5]. The GCN representations exhibit a clearer clustering by class, whereas the MLP representations remain more diffuse with greater overlap between classes. Because UMAP preserves local neighborhoods rather than global distances or cluster sizes, only the local clustering within each panel is informative.

[5] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. arXiv preprint, 2018. URL https://arxiv.org/abs/1802.03426

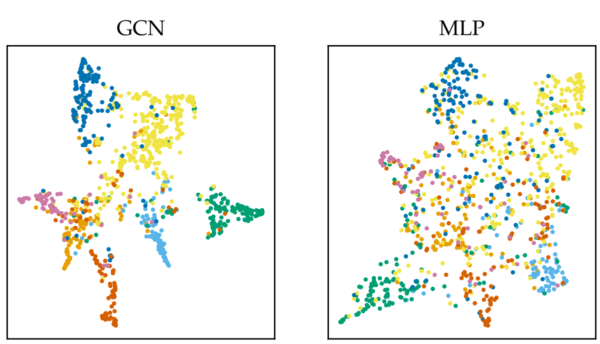


Figure 8.2: Two-dimensional UMAP projection of the test-node representations learned on Cora by GCN (left) and by the MLP (right), colored by class. Each panel is an independent projection; only the grouping within a panel is informative.

8.3 Link Prediction with GAE

The second implementation addresses link prediction. GAE (Section 7.2.2) and an MLP-based encoder with the same decoder are trained to predict co-purchase links on the Amazon Computers graph.

8.3.1 The Amazon Computers Dataset

Amazon Computers is a segment of the Amazon co-purchase graph [6]. Each node is a product, each edge is undirected and connects two products frequently bought together, and each node carries a bag-of-words vector extracted from product reviews. The task uses no node labels: the supervision is the edge set itself. Table 8.5 summarizes the dataset.

[6] Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of graph neural network evaluation. In *NeurIPS Workshop on Relational Representation Learning*, 2018. https://arxiv.org/abs/1811.05868

Property	Value
Nodes	13,752
Edges	245,861
Average degree	35.76
Node feature dimension	767
Split (train/val/test edges)	85% / 5% / 10%

Table 8.5: Summary of the Amazon Computers benchmark dataset.

The dataset is provided as a benchmark through PyTorch Geometric, so no additional cleaning is performed. The only transformation applied to the node features is a row-wise L1 normalization.

The split is defined over edges. A seeded sampling assigns 85% of the edges to training, 5% to validation, and 10% to test; validation and test edges are removed from the graph on which the encoder operates, so message passing runs on the training structure only. Both the validation and test sets are augmented with an equal number of sampled non-edges, the negative examples introduced in Section 7.2.1, so that evaluation is a binary classification over held-out pairs, with the non-edges drawn to be genuine — absent from the full graph and shared by no two evaluation sets. At this scale, the evaluation sets contain tens of thousands of pairs each, which keeps the variability of the resulting estimates low. The training setting remains transductive: all nodes and their features are visible throughout, while only the evaluation edges are held out from the training graph.

8.3.2 Model and Training

Table 8.6 specifies both models. The GAE encoder stacks two GCN layers, one as the hidden layer and the other as the output embedding layer; the MLP encoder consists of two hidden layers applied to each node independently, followed by an output embedding layer, resulting in about twice as many parameters as the GAE encoder. Both feed the same inner product decoder (7.6), which has no parameters of its own, so the encoder is what differs between the two models.

Component	GAE	MLP
Encoder	2 GCN layers	3 MLP layers
Hidden dimensions	64	128 / 64
Hidden activation	ReLU	
Embedding dimension	32	
Decoder	inner product	
Trainable parameters	51,232	108,640

Table 8.6: Model configuration for link prediction on Amazon Computers.

Table 8.7 specifies the optimization, identical for both models. Negative examples are resampled at every epoch, with one non-edge sampled per training edge. The loss is the binary cross-entropy between the reconstructed probabilities (7.7) and the observed adjacency over the sampled pairs, with no regularization. The choice of 400 epochs is deliberate: at that point both models are still improving slightly without clear signs of overfitting, as the loss curves in the next subsection confirm.

Parameter	Value
Loss	Binary cross-entropy
Optimizer	Adam
Learning rate	0.01
Weight decay	0
Dropout	0
Epochs	400
Batch size	full graph
Negative sampling	1:1, resampled per epoch

Table 8.7: Training configuration for link prediction on Amazon Computers, identical for both models.

The validation set is evaluated at every epoch, and the parameters achieving the highest validation AUC are retained. The results reported in the next subsection are obtained from the selected checkpoint evaluated on the test pairs.

8.3.3 Results

Figure 8.3 shows the reconstruction loss over the 400 epochs of training. Both models improve over training, on both the training and validation sets, and neither shows signs of overfitting: the validation loss follows the same decreasing trend as the training loss until the last epoch. The GAE loss remains below the MLP loss from early in training and stays there. Among the three tasks, this is the one in which the MLP is a genuine competitor: it learns the problem, so the comparison is between two models that both perform effectively.

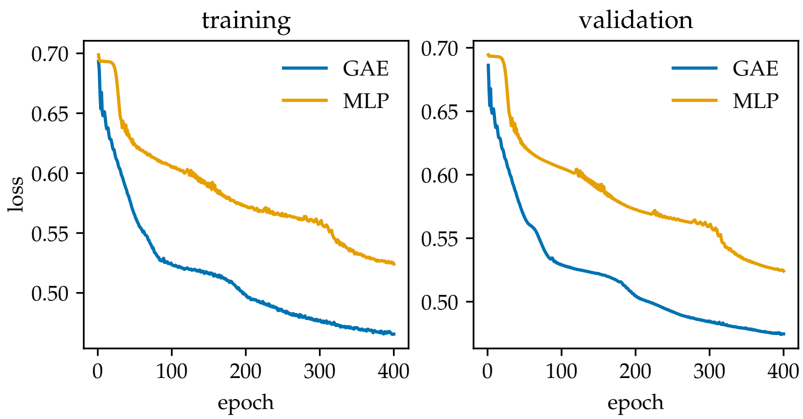


Figure 8.3: Training (left) and validation (right) loss per epoch on Amazon Computers for GAE and the MLP-based encoder. Each panel is scaled to its own data.

Two threshold-free metrics summarize performance over the held-out pairs. The AUC is the probability that a random true edge is scored above a random non-edge; the average precision (AP) summarizes the precision–recall curve, giving more importance to retrieving true edges among the highest-scored pairs. Table 8.8 reports them.

Metric	Model	Median	Mean	Std
AUC	GAE	0.9389	0.9352	0.0077
	MLP	0.8778	0.8567	0.0297
AP	GAE	0.9372	0.9346	0.0071
	MLP	0.8741	0.8589	0.0248

Table 8.8: Link prediction on Amazon Computers, over ten seeds.

Figure 8.4 shows the two ROC curves on the test pairs. The GAE curve lies above the MLP curve, with the largest separation at low false-positive rates. In this regime, the GAE achieves higher true-positive rates for the same false-positive rate, indicating better discrimination between true and false links.

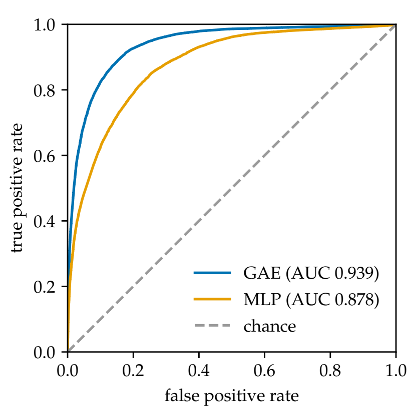


Figure 8.4: ROC curves on the Amazon Computers test pairs for GAE and the MLP-based encoder, with each AUC annotated; the dashed diagonal is the performance of a random score.

Although link prediction scores are often used to identify the most likely candidate links, enumerating the top co-purchases would add nothing to the comparison pursued here.

The GAE reaches a median AUC of 0.9389 and a median AP of 0.9372, compared with 0.8778 and 0.8741 for the MLP, an advantage of approximately 6 percentage points on both metrics, with roughly half the parameters. The margin is the smallest of the three tasks, and the reason is instructive: the product features alone provide substantial predictive signal for co-purchase, so an encoder that ignores the graph already ranks pairs well. Message passing improves the ranking further by encoding structural information from neighboring nodes into the embeddings.

8.4 Graph Classification with GIN

The third implementation addresses graph classification. GIN (Section 7.3.2) and an MLP are trained to predict whether a chemical compound is active or inactive in the NCI1 benchmark.

8.4.1 The NCI1 Dataset

NCI1 is a benchmark of chemical compounds from the National Cancer Institute that were screened for activity against non-small cell lung cancer [7]. Each graph is a compound: its nodes are atoms, each carrying a one-hot vector that encodes the atom type; each edge is undirected and represents a chemical bond between two atoms; and each graph is labeled active or inactive, with the two classes in near-equal proportion. Table 8.9 summarizes the dataset.

[7] Nikil Wale, Ian A. Watson, and George Karypis. Comparison of descriptor spaces for chemical compound retrieval and classification. *Knowledge and Information Systems*, 14(3):347–375, 2008. DOI: 10.1007/s10115-007-0103-5

Property	Value
Graphs	4,110
Average nodes per graph	29.87
Average edges per graph	32.3
Average degree	2.16
Node feature dimension	37
Classes	2
Class balance	2,053 / 2,057
Split (train/val/test graphs)	80% / 10% / 10%

Table 8.9: The NCI1 dataset as served by PyTorch Geometric.

The dataset is provided as a benchmark through PyTorch Geometric, so no additional cleaning is performed. No additional transformation is required, since node features are already one-hot encoded.

The split is defined over graphs. A seeded, stratified sampling assigns 80% of the graphs to training, 10% to validation, and 10% to test, preserving the class proportions in each subset. This allocation balances the need to maximize the number of graphs available for parameter learning with the need to retain evaluation subsets large enough to reduce the variability of the resulting estimates. The setting is inductive (Section 6.4): the graphs on which the model is evaluated are disjoint from those on which it is trained, and the shared parameters of the message-passing layers are what allow the model to generalize to graphs it has never seen.

8.4.2 *Model and Training*

Table 8.10 specifies both models. GIN stacks three layers of the update (7.9), each with a two-layer MLP and a fixed $\epsilon = 0$ as its update function; the MLP baseline applies three layers to every node independently, which gives it about twice as many trainable parameters as GIN. Each update function applies batch normalization [8] before the activation to stabilize the intermediate representations during training; the MLP baseline applies the same normalization between its layers, so this component is also shared by both models. Both models are then followed by the shared sum readout (7.8) and the same classification head, making the graph-level pipeline identical except for the message passing.

[8] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015. https://arxiv.org/abs/1502.03167

Table 8.10: Model configuration for graph classification on NCI1.

Component	GIN	MLP
Layers	3 GIN layers	3 MLP layers
Hidden dimensions	64	256 / 128 / 64
$\epsilon^{(k)}$	0 (fixed)	—
Normalization	batch normalization	
Hidden activation	ReLU	
Readout	sum	
Classification head	1 linear	
Output dimension	2	
Trainable parameters	24,130	51,906

Table 8.11 specifies the optimization, identical for both models. Both models produce class probabilities through a softmax transformation and are trained using cross-entropy loss with no regularization. Training uses a mini-batch size of 32, since the dataset consists of many independent graphs rather than a single graph.

The validation set is evaluated at every epoch, and the parameters achieving the highest validation accuracy are retained. The results reported in the next subsection are obtained from the selected checkpoint evaluated on the test graphs.

Parameter	Value
Loss	Cross-entropy
Optimizer	Adam
Learning rate	0.01
Weight decay	0
Dropout	0
Epochs	200
Batch size	32

Table 8.11: Training configuration for graph classification on NCI1, identical for both models.

8.4.3 *Results*

Figure 8.5 shows the loss over training. The training dynamics differ substantially between the two models. The GIN training loss decreases steadily throughout training. Its validation loss reaches a minimum after roughly 50 epochs and then increases gradually while becoming more variable. This rise does not by itself indicate a loss of classification performance: the cross-entropy grows when the model becomes more confident on the graphs it misclassifies. Since model selection is based on validation accuracy rather than on validation loss, training is not stopped at that minimum, and the retained checkpoint is the one that classifies the validation graphs best. In contrast, the MLP training loss decreases for only a few epochs before flattening near 0.62, where it remains for the rest of training; its validation loss exhibits the same behavior. The absence of a gap between training and validation loss indicates that the MLP does not overfit; instead, its early plateau is consistent with underfitting, suggesting that the available node features alone are insufficient for further improvement under this architecture.

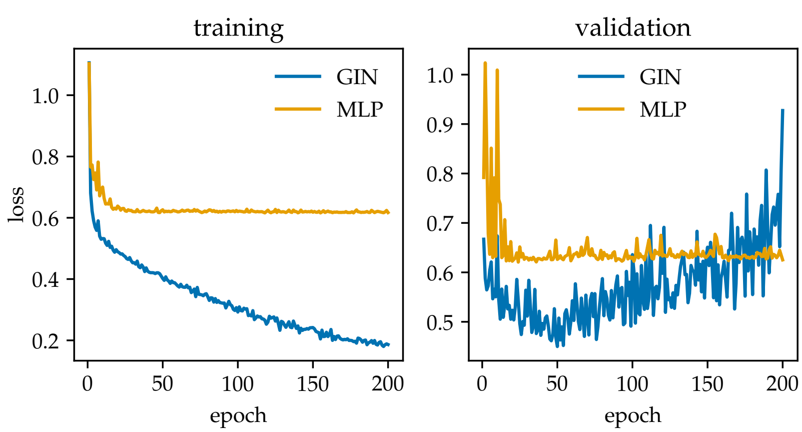


Figure 8.5: Training (left) and validation (right) loss per epoch on NCI1 for GIN and the MLP. Each panel is scaled to its own data.

Performance is measured by accuracy over the held-out graphs. Accuracy provides a global measure of classification effectiveness, while the confusion matrices complement this metric by showing how errors are distributed across classes. This reveals potential differences in classification behavior between active and inactive compounds. Table 8.12 reports the test accuracy across ten seeds.

Model	Median	Mean	Std
GIN	0.8199	0.8168	0.0166
MLP	0.6496	0.6477	0.0277

Table 8.12: Test accuracy on NCI1, over ten seeds.

GIN reaches a median accuracy of 0.8199, compared with 0.6496 for the MLP, an advantage of approximately 17 percentage points despite using roughly half the parameters. The gap reflects the information available to each architecture. With one-hot atom features and no access to connectivity before the readout, the structure-blind MLP can distinguish graphs only through their node attributes; graphs with the same multiset of node features receive identical representations regardless of their connectivity. GIN, in contrast, aggregates neighborhood information before the graph-level readout, allowing its representations to incorporate structural patterns. The results show that message passing provides an advantage when there is an informative underlying graph structure.

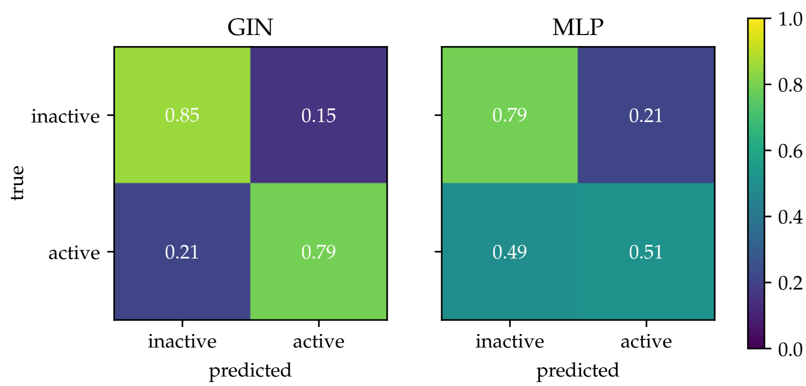


Figure 8.6: Row-normalized confusion matrices on the NCI1 test graphs for GIN (left) and the MLP (right), under a shared color scale.

Figure 8.6 decomposes the errors. GIN classifies both active and inactive compounds correctly in approximately four out of five cases. The MLP achieves substantially better performance on inactive compounds than on active ones, for which its predictions are close to chance. This behavior is consistent with the limitations of a structure-blind model when the task requires information beyond node attributes.

9 Limitations of the Message-Passing Framework

Contents

The message-passing framework has been presented in the setting where it succeeds. This chapter turns to its limits, and two kinds of limit are distinguished. The first concerns propagation — what happens to representations as messages are passed and when the homophily assumption behind aggregation no longer holds. The second concerns expressive power — what the framework can distinguish at the graph level, independent of training.

9.1 Pathologies of Neighborhood Propagation

This section examines three ways in which neighborhood propagation can degrade the representations it produces: *oversmoothing*, the collapse of representations under depth; *oversquashing*, the distortion of information drawn from far away; and *heterophily*, where neighboring nodes tend to be dissimilar rather than similar.

9.1.1 Oversmoothing

A GCN layer smooths, reducing variation across edges and making representations of neighboring nodes more similar (Section 7.1.2). A few layers may be sufficient to align the representations with the labels on a homophilous graph; the difficulty is that smoothing does not stop at a useful point. Applied repeatedly, the operation that makes adjacent nodes more similar makes all nodes similar: as depth grows, the representations within each connected component converge toward a common vector. This is *oversmoothing*, first identified for the GCN as a consequence of its Laplacian smoothing [1].

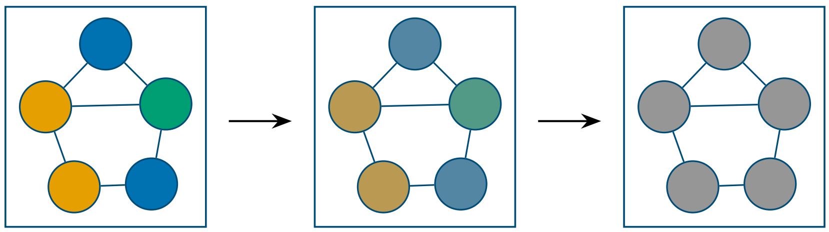


Figure 9.1: Node representations at increasing depth. With few layers the representations of different nodes remain distinct; as layers are stacked, repeated smoothing drives them together, until at large depth the nodes of each connected component converge toward a common representation and can no longer be told apart.

This places a ceiling on useful depth that is in direct tension with the receptive field of Section 6.3. The useful depth of a network is therefore a compromise between the receptive field and oversmoothing. Architectural designs that relieve this tension are revisited in Chapter 10.

[1] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3538–3545, 2018. DOI: 10.1609/aaai.v32i1.11604

9.1.2 *Oversquashing*

The receptive field grows with depth, suggesting that a deep enough network lets any node draw on any other. However, the number of nodes within k hops typically grows exponentially with k, yet the representation summarizing this region is a vector of fixed dimension, the same size whether it gathers a handful of nodes or thousands. Information from an exponentially growing region is compressed into a fixed-size representation, making information loss unavoidable.

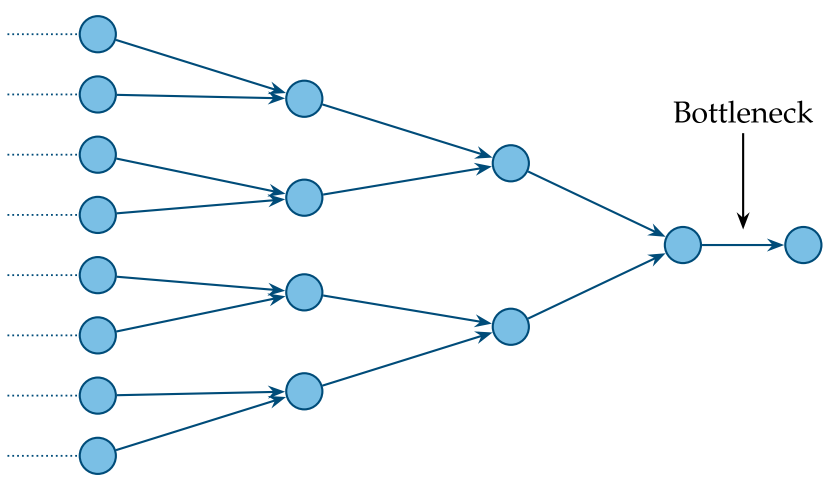


Figure 9.2: The receptive field of a node, unrolled over layers, typically grows exponentially: each additional hop multiplies the number of nodes that must be summarized. The representation gathering this region is a vector of fixed size, independent of how many nodes feed into it, so information from the wider neighborhood is compressed, reducing the contribution of any single distant node.

This is *oversquashing* (Figure 9.2): the distortion that results when information from a node's wider neighborhood is squeezed through the fixed-size representations of the intermediate nodes [2]. Its consequence is that message-passing networks struggle on tasks depending on long-range interaction. To reach a distant target, a signal must pass through every intermediate representation, and at each step it is aggregated with everything else in that node's neighborhood. The framework can reach distant nodes by adding depth, but the information it gathers from them can arrive too degraded to be useful.

[2] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *Proceedings of the 9th International Conference on Learning Representations*, 2021. https://arxiv.org/abs/2006.05205

Oversquashing and oversmoothing both worsen with depth but are distinct failures. Oversmoothing is a loss of distinction *between nodes*: their representations converge. Oversquashing is a loss of information *from distance*: one node's representation cannot faithfully hold its large receptive field. Together they bound how deep a network can usefully be made.

9.1.3 Heterophily

Oversmoothing and oversquashing are costs of the mechanism that arise whatever the data. Heterophily is instead a failure of fit between mechanism and data. A GCN layer rests on the assumption that adjacent nodes are similar — the homophily of Section 3.3. When it holds, aggregating a neighborhood reinforces what a node indicates; when it fails, the same aggregation works against the task.

The degree to which a graph is homophilous can be measured, for a node-classification task, by the fraction of edges joining nodes of the same class. Writing y_i for the class label of node i, the *edge homophily* of a graph is

$$h(\mathcal{G}) = \frac{\left|\{\,\{i,j\} \in E : y_i = y_j\,\}\right|}{m}, \tag{9.1}$$

the number of edges whose endpoints share a label divided by the total number of edges m. A value near one describes a strongly homophilous graph; a value near zero, a strongly heterophilous one, in which edges predominantly join nodes of different classes.

On a heterophilous graph, the aggregation misleads: a node's neighbors mostly belong to other classes, so smoothing their representations pulls the node toward classes it does not belong to, blurring the distinction the task requires. The framework does not fail to use the graph; it uses the graph through an assumption the graph does not satisfy. Aggregation schemes that separate a node's own contribution from its neighbors', or learn to weight neighbors unequally, relax this assumption and are noted in Chapter 10.

9.2 *The Expressivity Ceiling under 1-WL*

Section 6.5 established that the expressive power of message passing is bounded above by the 1-WL test, and that the architecture of Section 7.3.2 was designed to reach this ceiling. This section makes the limit concrete by exhibiting what it excludes.

What the ceiling excludes is a pair of non-isomorphic graphs that 1-WL — and therefore every message-passing network — colors identically. The simplest examples arise among pairs of *regular* graphs, in which every node has the same degree. All nodes begin with the same color; and whenever every node shares a color, each one has the same number of identically colored neighbors, so the refinement assigns them all the same new color. The condition therefore reproduces itself at every round, and the test never separates the nodes, so two such graphs receive the same final coloring. Yet regular graphs of the same degree need not be isomorphic, differing in structural properties the count of same-colored neighbors never exposes, such as whether they form one connected cycle or two disjoint shorter ones (Figure 9.3).

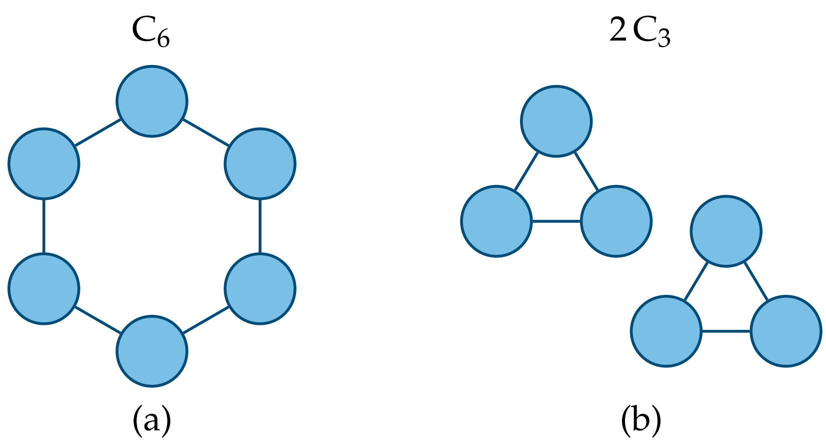


Figure 9.3: Two non-isomorphic regular graphs in which every node has the same degree. The 1-WL test colors all nodes of both graphs identically and never separates them, so no message-passing network can tell the two graphs apart, although they are not isomorphic.

This is a genuine ceiling, not a shortcoming of present architectures, because it follows from the locality of the computation: a network that only ever sees aggregated neighborhoods cannot distinguish nodes whose neighborhoods are themselves indistinguishable. Surpassing it means changing what the computation may see. Such methods fall outside the present framework; they are taken up as future work in Chapter 10.

10 Conclusions and Future Directions

Contents

This closing chapter looks back over what was established and forward to what it opens. The first part gathers the contributions of the work and weighs them against the objectives set out in the introduction. The second part maps the directions that the framework, once understood together with its limits, makes available for further study.

10.1 *Synthesis of Contributions*

The general objective stated in Section 1.2 was to provide a self-contained reference on GNNs for homogeneous graphs under the message-passing framework: a single document that develops the theory, presents representative architectures, and evaluates them through reproducible implementations. The work meets this objective, and its specific aims were addressed in turn across the document.

The foundations that motivate GNNs were established first. Graph-structured data was characterized as the natural description of relational domains, and the properties that set it apart from tabular, grid, and sequential data — variable size, absence of a canonical ordering, irregular local structure — were shown to be precisely the properties that a standard neural network cannot accommodate. The MLP was examined as the concrete case: its reliance on a fixed input dimension and a fixed coordinate ordering makes it unable to operate on graphs without discarding the very structure that carries the signal. From this failure the requirements on any graph model were derived, with permutation invariance and equivariance playing a central role.

On these foundations the message-passing framework was developed as the unifying formulation. A node's representation was defined through the repeated aggregation of information from its neighborhood, decomposed into the message, aggregation, and update functions, and shown to satisfy the invariance and equivariance requirements by construction. The framework was then examined through its principal implications: the receptive field that successive layers induce, the distinction between the transductive and inductive settings, the expressive power of the resulting models and its ceiling, and the design space that the three functions leave open. The MLP reappeared within this space as the degenerate case of a graph with no relations, whose nodes are all independent, locating it as a particular case of the same framework rather than a separate object.

The three representative architectures were then derived as instances of the framework, one for each standard level of graph learning. GCN was presented for node classification, GAE for link prediction through the encoder–decoder view, and GIN for graph classification, with injective aggregation motivated as the choice that reaches the expressive ceiling the framework permits. Each architecture was obtained by specializing the message, aggregation, and update functions, placing them in the single design space.

The architectures were implemented on standard benchmarks — Cora for GCN, Amazon Computers for GAE, and NCI1 for GIN — in PyTorch Geometric, with reproducible code in a companion repository. Each was evaluated against a MLP baseline that assumes no graph structure and predicts from node features alone. Across all three tasks the architecture that uses the graph improves on the baseline that does not, confirming in practice the predictive benefit that the theory anticipates: when the data has an underlying graph structure, the relational structure carries information, and a model built to exploit it extracts signal that a feature-only model cannot.

Finally, the limitations of the framework were examined on their own: oversmoothing, oversquashing, heterophily, and the ceiling on expressive power, highlighting the mechanisms that give rise to them and their implications for the design and performance of these models.

The contribution of the work is the reference document itself. Its value lies not in any single result but in the assembly: a path from the motivation for learning on graphs to the limits of the message-passing framework, with the theory, architectures, and their evaluation developed in a consistent notation and continuous line of argument, intended to serve as a foundational resource for the study of the field.

10.2 Directions for Future Research

The scope of this work was deliberately restricted to homogeneous, undirected, simple graphs and to three architectures within the message-passing framework. Its boundaries mark where the field continues, with future research extending across the limitations of the framework, unexplored regions of its design space, new domains and tasks, and architectures that move beyond message passing.

10.2.1 Addressing the Limitations of the Framework

Chapter 9 established the limitations of message passing as consequences of its mechanism. Each has motivated a direction of research aimed at overcoming it.

The ceiling on useful depth imposed by oversmoothing is addressed by architectures that give a node access to its earlier, less smoothed representations rather than only the most recent one, so that depth can grow without the representations collapsing toward a common vector [1]. Oversquashing, the distortion of information from distant parts of the graph as it is squeezed through intermediate nodes, is addressed by mitigating the associated bottleneck; one direction replaces local propagation with global attention mechanisms, a direction discussed separately below. Heterophily is met by aggregation schemes that no longer assume neighbors are similar: separating a node's own representation from its neighbors' and combining information across multiple rounds of neighborhood are designs identified as effective precisely where the homophily assumption fails [2]. The expressivity ceiling set by the 1-WL test is raised by computations that refine over tuples of nodes rather than single nodes, forming a higher-order hierarchy that goes beyond this bound [3].

[1] Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Ken-ichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. https://arxiv.org/abs/1806.03536

[2] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Advances in Neural Information Processing Systems 33*, 2020. https://arxiv.org/abs/2006.11468

[3] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019. DOI: 10.1609/aaai.v33i01.33014602

10.2.2 Unexplored Regions of the Design Space

This work examined a limited part of the design space the framework defines. Other parts of that space contain established architectures it did not develop.

The aggregation step was confined to sum, mean, and max. Replacing it with a learned, attention-based weighting of neighbors, so that a node determines how much each neighbor contributes, yields *Graph Attention Networks (GAT)* [4]. Separating the central node's own representation and learning to aggregate from a sampled subset of the neighborhood yields *GraphSAGE*, an architecture built for large graphs [5]. Both remain within message passing while exploring regions of its design space, such as learned aggregation and neighborhood sampling.

[4] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *Proceedings of the 6th International Conference on Learning Representations*, 2018. https://arxiv.org/abs/1710.10903

[5] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems 30*, 2017. https://arxiv.org/abs/1706.02216

A further region opens once the graph is no longer static. Where the structure or the node features evolve in time, the framework can be extended to couple message passing over the graph with a model of temporal change, as in spatio-temporal architectures for forecasting on graphs whose signals evolve over time [6].

[6] Bing Yu, Haoteng Yin, and Zhanxing Zhu. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, pages 3634–3640, 2018. DOI: 10.24963/ijcai.2018/505

10.2.3 *Beyond Homogeneous Graphs and Standard Tasks*

Heterogeneous graphs carry several types of nodes and edges. Modeling them requires machinery the homogeneous setting does not: transformations specific to each relation type, so that a connection between two kinds of entity is treated differently from a connection between two others [7]. The homogeneous case developed in this work is the prerequisite for these models, which extend message passing to a typed structure.

A different extension changes the unit of prediction. The three tasks of this work attach predictions to nodes, edges, or whole graphs; a subgraph — a portion of a graph with its own internal structure and position within the larger graph — is none of these, and learning representations for subgraphs as primary objects supports tasks not captured by the standard levels [8]. Further still, the models considered here are discriminative, predicting properties of a given graph; another direction is graph generation, where a model learns a distribution over graph structures and samples new instances from it. This generative setting has applications in domains such as molecular design [9].

[7] Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web — 15th International Conference, ESWC 2018*, volume 10843 of *Lecture Notes in Computer Science*, pages 593–607, 2018. DOI: 10.1007/978-3-319-93417-4_38

[8] Emily Alsentzer, Samuel G. Finlayson, Michelle M. Li, and Marinka Zitnik. Subgraph neural networks. In *Advances in Neural Information Processing Systems 33*, 2020. https://arxiv.org/abs/2006.10538

[9] Jiaxuan You, Rex Ying, Xiang Ren, William L. Hamilton, and Jure Leskovec. GraphRNN: Generating realistic graphs with deep auto-regressive models. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. https://arxiv.org/abs/1802.08773

10.2.4 *Graph Transformers*

The directions above remain within message passing or extend it. A final direction steps outside it. The transformer replaces local neighborhood-based propagation in message passing with global attention: every node attends directly to every other in a single operation, regardless of distance on the graph [10]. Graph structure, no longer acting as the channel through which information flows, is instead incorporated as a positional or structural encoding added to this global attention mechanism. This mitigates the bottleneck associated with oversquashing — a distant node is now reachable in a single attention step rather than many hops — at the cost of the locality that gives message passing its scalability and inductive bias.

These directions extend the message-passing framework by relaxing assumptions such as locality, expressivity limits, or graph homogeneity, as well as applying the same foundations to new domains, tasks, and generative settings. The framework developed in this work is therefore not an endpoint but a starting point, providing the basis from which these extensions and applications can be understood and evaluated.

[10] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform badly for graph representation? In *Advances in Neural Information Processing Systems 34*, 2021. https://arxiv.org/abs/2106.05234

Bibliography

Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. In *Proceedings of the 9th International Conference on Learning Representations*, 2021. https://arxiv.org/abs/2006.05205.

Emily Alsentzer, Samuel G. Finlayson, Michelle M. Li, and Marinka Zitnik. Subgraph neural networks. In *Advances in Neural Information Processing Systems 33*, 2020. https://arxiv.org/abs/2006.10538.

Peter W. Battaglia, Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, Caglar Gulcehre, Francis Song, Andrew Ballard, Justin Gilmer, George Dahl, Ashish Vaswani, Kelsey Allen, Charles Nash, Victoria Langston, Chris Dyer, Nicolas Heess, Daan Wierstra, Pushmeet Kohli, Matt Botvinick, Oriol Vinyals, Yujia Li, and Razvan Pascanu. Relational inductive biases, deep learning, and graph networks. arXiv preprint, 2018. URL https://arxiv.org/abs/1806.01261.

Christopher M. Bishop and Hugh Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. ISBN 978-3-031-45467-7.

Fan R. K. Chung. *Spectral Graph Theory*, volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, 1997. ISBN 978-0-8218-0315-8.

Reinhard Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, 5th edition, 2017. ISBN 978-3-662-53621-6.

Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. https://arxiv.org/abs/1903.02428.

Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, 2017. https://arxiv.org/abs/1704.01212.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. ISBN 978-0-262-03561-3.

William L. Hamilton. *Graph Representation Learning*, volume 46 of *Synthesis Lectures on Artificial Intelligence and Machine Learning*. Morgan & Claypool, 2020. ISBN 978-1-68173-965-6.

William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems 30*, 2017. https://arxiv.org/abs/1706.02216.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015. https://arxiv.org/abs/1502.03167.

John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596 (7873):583–589, 2021. DOI: 10.1038/s41586-021-03819-2.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, 2015. https://arxiv.org/abs/1412.6980.

Thomas N. Kipf and Max Welling. Variational graph auto-encoders. In *NIPS Workshop on Bayesian Deep Learning*, 2016. https://arxiv.org/abs/1611.07308.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *Proceedings of the 5th International Conference on Learning Representations*, 2017. https://arxiv.org/abs/1609.02907.

Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3538–3545, 2018. DOI: 10.1609/aaai.v32i1.11604.

Rodrigo Emmanuel Macias Pantoja. When GNNs Beat MLPs. GitHub repository, 2026. URL https://github.com/roymacias/when-gnns-beat-mlps.

Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. arXiv preprint, 2018. URL https://arxiv.org/abs/1802.03426.

Miller McPherson, Lynn Smith-Lovin, and James M. Cook. Birds of a feather: Homophily in social networks. *Annual Review of Sociology*, 27 (1):415–444, 2001. DOI: 10.1146/annurev.soc.27.1.415.

Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019. DOI: 10.1609/aaai.v33i01.33014602.

Christopher Morris, Yaron Lipman, Haggai Maron, Bastian Rieck, Nils M. Kriege, Martin Grohe, Matthias Fey, and Karsten Borgwardt. Weisfeiler and Leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023. https://jmlr.org/papers/v24/22-0240.html.

Michael Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *The Semantic Web — 15th International Conference, ESWC 2018*, volume 10843 of *Lecture Notes in Computer Science*, pages 593–607, 2018. DOI: 10.1007/978-3-319-93417-4_38.

Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. Collective classification in network data. *AI Magazine*, 29(3):93–106, 2008. DOI: 10.1609/aimag.v29i3.2157.

Oleksandr Shchur, Maximilian Mumme, Aleksandar Bojchevski, and Stephan Günnemann. Pitfalls of graph neural network evaluation. In *NeurIPS Workshop on Relational Representation Learning*, 2018. https://arxiv.org/abs/1811.05868.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15 (56):1929–1958, 2014. https://jmlr.org/papers/v15/srivastava14a.html.

Jonathan M. Stokes, Kevin Yang, Kyle Swanson, Wengong Jin, Andres Cubillos-Ruiz, Nina M. Donghia, Craig R. MacNair, Shawn French, Lindsey A. Carfrae, Zohar Bloom-Ackermann, Victoria M. Tran, Anush

Chiappino-Pepe, Ahmed H. Badran, Ian W. Andrews, Emma J. Chory, George M. Church, Eric D. Brown, Tommi S. Jaakkola, Regina Barzilay, and James J. Collins. A deep learning approach to antibiotic discovery. *Cell*, 180(4):688–702, 2020. DOI: 10.1016/j.cell.2020.01.021.

Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *Proceedings of the 6th International Conference on Learning Representations*, 2018. https://arxiv.org/abs/1710.10903.

Nikil Wale, Ian A. Watson, and George Karypis. Comparison of descriptor spaces for chemical compound retrieval and classification. *Knowledge and Information Systems*, 14(3):347–375, 2008. DOI: 10.1007/s10115-007-0103-5.

Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. Simplifying graph convolutional networks. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. https://arxiv.org/abs/1902.07153.

Keyulu Xu, Chengtao Li, Yonglong Tian, Tomohiro Sonobe, Kenichi Kawarabayashi, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. https://arxiv.org/abs/1806.03536.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *Proceedings of the 7th International Conference on Learning Representations*, 2019. https://arxiv.org/abs/1810.00826.

Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. Do transformers really perform badly for graph representation? In *Advances in Neural Information Processing Systems 34*, 2021. https://arxiv.org/abs/2106.05234.

Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 974–983, 2018. DOI: 10.1145/3219819.3219890.

Jiaxuan You, Rex Ying, Xiang Ren, William L. Hamilton, and Jure Leskovec. GraphRNN: Generating realistic graphs with deep autoregressive models. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. https://arxiv.org/abs/1802.08773.

Bing Yu, Haoteng Yin, and Zhanxing Zhu. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, pages 3634–3640, 2018. DOI: 10.24963/ijcai.2018/505.

Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. Beyond homophily in graph neural networks: Current limitations and effective designs. In *Advances in Neural Information Processing Systems 33*, 2020. https://arxiv.org/abs/2006.11468.

Marinka Zitnik, Monica Agrawal, and Jure Leskovec. Modeling polypharmacy side effects with graph convolutional networks. *Bioinformatics*, 34(13):i457–i466, 2018. DOI: 10.1093/bioinformatics/bty294.